Department of Electronics & Communication Engineering

Internet of Things Lab

VI Semester (22ECL671)

Staffs in-charge:

Prof. Vidyashree K N
Prof. Aishwarya N

Lab Instructors: Smt. Gayatri & Smt. Veena

HOD: Dr. Shobha K R

Name of the Student	:	
Semester and Section	:	
USN	:	
Batch	:	

Dayananda Sagar College of Engineering

Shavige Malleshwara Hills, Kumaraswamy Layout,
Banashankari, Bangalore-560111, Karnataka
Tel : +91 80 26662226 26661104 Extn : 2731 Fax : +90 80 2666 0789
Web - http://www.dayanandasagar.edu Email : hod-ece@dayanandasagar.edu
(An Autonomous Institute Affiliated to VTU, Approved by AICTE & ISO 9001:2008 Certified)

About the College

The Dayananda Sagar College of Engineering was established in 1979 by Sri R. Dayananda Sagar. Dayananda Sagar College of Engineering operates under the aegis of the **Mahatma Gandhi Vidya Peetha Trust** is approved by All India Council for Technical Education (AICTE), Govt. of India and affiliated to Visvesvaraya Technological University. It has the widest choice of engineering branches having 20 Under Graduate courses & 6 Post Graduate courses. In addition, it has 20 Research Centres in different branches of Engineering catering to research scholars for obtaining Ph.D. under VTU. Various courses are accredited by NBA.

The Institute is spread over 23 acres of land with large infrastructure supported by laboratories with state-of-the-art, Equipment & Machines. The Central Library with modern facilities and the Digital Library provides the knowledge base for the students.

The campus is WI-FI equipped with large bandwidth internet facility. The College has good faculty strength with highest professional integrity and committed to the academics with transparency in their actions. Each faculty takes the responsibility of mentoring a certain number of students through personal attention paving the way for the students' professional growth. The faculty are research oriented having number of sponsored R & D projects from different agencies such as Department of Science & Technology, Defence R & D organizations, Indian Space Research Organization, AICTE etc.

About the Department

Department of Electronics and Communication Engineering is one of the oldest and biggest department in the DSI group with 50 faculty members and 1200+ students. Most of our faculty have pursued Ph.D. and others are in the process. The ECE department provides high-quality, innovative technical education at the undergraduate, graduate, and research levels. At present, the department offers an UG course (BE) with an intake of 240, a PG course in VLSI Design and Embedded Systems with an intake of 18 students and Ph.D. The department has got an excellent infrastructure with sophisticated labs, class rooms and R & D center. The department faculty and support personnel are dedicated towards helping students achieve success in today's world by offering them the knowledge and skills required. The department places a strong emphasis on leading-edge research through its research centre and research forum. These factors together enable our graduating students to create a big impact on the workforce from day one.

In addition to offering innovative courses, the department is more focused on improving students' educational experiences by offering value-added courses, skill-development programs, technical challenges and hackathons at state, national, and international arenas. Through its professional societies and technical clubs, the department takes great satisfaction in exposing its students to current industry developments and upcoming technologies. The dedicated staff at the ECE department help students procure jobs in prestigious companies both domestically and abroad.

Vision of the College:

To impart quality technical education with a focus on Research and Innovation emphasizing on Development of Sustainable and Inclusive Technology for the benefit of society.

Mission of the College:

- To provide an environment that enhances creativity and Innovation in pursuit of Excellence.
- To nurture teamwork in order to transform individuals as responsible leaders and entrepreneurs.
- To train the students to the changing technical scenario and make them to understand the importance of Sustainable and Inclusive technologies.

Vision of the Department

To achieve continuous improvement in quality technical education for global competence with focus on industry, societal needs, research and professional success.

Mission of the Department

- Offering quality education in Electronics and Communication Engineering with effective teaching learning process in multidisciplinary environment.
- Training the students to take-up projects in emerging technologies and work with team spirit.
- To imbibe professional ethics, development of skills and research culture for better placement opportunities.

Program Educational Objectives [PEOs]

The Graduate students must be able to:
PEO-1: Ready to apply the state-of-art technology in industry and meeting the societal need with Knowledge of Electronics and Communication Engineering due to strong academic culture.
PEO-2: Competent in technical and soft skills to be employed with capability of working in Multidisciplinary domains.
PEO-3: Professionals, capable of pursuing higher studies in technical, research management programs.

Programme Specific Outcomes [PSOs]

PSO-1: Design, develop and integrate electronic circuits and systems using current practices and Standards.
PSO-2: Apply knowledge of hardware and software in designing Embedded and Communication Systems.

Programme Outcomes [POs]

Engineering Graduates will be able to:

1. **Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature and analyse complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
3. **Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
4. **Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
5. **Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
6. **The Engineer and The World:** Analyse and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture, and environment.
7. **Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.
8. **Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9. **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10. **Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11. **Life-Long Learning:** Recognize the need for, and have the preparation and ability for
    i) independent and life-long learning
    ii) adaptability to new and emerging technologies and
    iii) critical thinking in the broadest context of technological change

Internet of Things Lab (Standalone ABNC Lab)

Course Code: 22ECL671 **CIE Marks:** 50 **Credits:** 1
L : T : P : 0 : 0 : 1 **Lab Duration**: 02 Hrs **Total:** 12 Lab sessions

COURSE OBJECTIVES:

1.	Develop skills in interfacing sensors and actuators with ESP32 for real-time data acquisition.
2.	Explore IoT communication protocols and networking concepts using ESP32.
3.	Implement IoT-based web servers and cloud integrations for remote monitoring and control.
4.	Design and develop smart IoT applications with automation, security, and efficient data handling.

COURSE OUTCOMES: At the end of the course, the student will be able to

CO1	Demonstrate the ability to interface sensors and actuators with ESP32
CO2	Design and implement programs for wireless communication and networking of IoT devices using suitable techniques
CO3	Design and Implement IoT-based web servers for monitoring control of devices
CO4	Develop real-world IoT applications with remote monitoring and control capabilities

Mapping of Course Outcomes to Program Outcomes:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	2	1	2	-	2	1	-	-	-	-	-	-	2
CO2	2	1	2	-	2	1	-	-	-	-	-	-	2
CO3	2	1	2	-	2	-	-	-	-	-	-	-	2
CO4	2	1	3	-	2	-	-	1	1	1	2	-	2

Weightage values: 3 – Strongly matched, 2 – moderately matched, 1 – weakly matched, (---) not matched.

List of Experiments

Sl. No.	Experiment Name	COs
1	a) Design an interface circuit to control LED light using ESP32. b) Design an interface circuit to control Buzzer using ESP32.	CO1
2	Design an interface circuit for DHT11 (Temperature and Humidity) Sensor with ESP32. Write a Program to display Temperature and Humidity on Serial Monitor.	CO1, CO2
3	Design an interface circuit with 16x2 I2C LCD AND ESP32 to display characters on LCD.	CO1
4	Design an interface circuit for Ultrasonic Sensor (HC-SR04) with ESP32. Write a program to measure the distance and display on Serial Monitor.	CO1, CO2
5	Design an interface circuit for Soil Moisture sensor with ESP32 to measure soil moisture and display it on a Serial monitor.	CO1, CO2
6	Design an interface circuit to detect the motion using IR sensor and turn on Buzzer upon detection.	CO1, CO2
7	Design an interface circuit for Pulse Sensor with ESP32. Write a Program to display BPM value on serial monitor.	CO1, CO2
8	a) Configure/Set your ESP32 development broad as an access point. b) Scan Wi-Fi networks and get Wi-Fi strength using ESP32 development board. c) Establish connection between IoT node and access point and display of local IP and gateway IP.	CO3, CO4
9	Design and implementation of HTTP based IoT web server using ESP32 to control the status of LED.	CO3, CO4
10	Design and implementation of HTTP based IoT web server using ESP32 to display DHT11 sensor value.	CO3, CO4
11	Design an interface for DHT11 (Temperature and Humidity) Sensor with ESP32. Write Program to display temperature and humidity on Blynk App. (Using Blynk Server).	CO2, CO3, CO4
12	Open Ended Experiment	CO1,2,3,4

DEPARTMENT of ELECTRONICS AND COMMUNICATION ENGINEERING

INTERNET OF THINGS LABORATORY

Index Sheet

Dayananda Sagar College of Engineering

Dept. of Electronics & Communication Engineering

Name of the Laboratory	**:**	Internet of Things Lab (Standalone ABNC Lab)
Semester	**:**	VI
No. of students /batch	**:**	22
No. of equipment's	**:**	22
Major Equipment's	**:**	ESP32 Development kit, Computer, Arduino Software
Operating System	**:**	Windows 10 and higher
Simulation Tools	**:**	Arduino IDE
Area of the lab in Sq. mts	**:**	102 Sq. mts
Lab in-charges	**:**	Prof. Vidyashree K N Prof. Aishwarya N
Instructor name	**:**	Smt. Gayatri Smt. Veena
HoD	**:**	Dr. Shobha K R

DAYANANDA SAGAR COLLEGE OF ENGINEERING
DEPARTMENT OF ELCTRONICS & COMMUNICATION ENGINEERING

DO's

- All the students should come to LAB on time with proper dress code and identity cards.
- Keep your belongings in the bag rack of laboratory.
- Students have to enter their name, USN, time-in/out and signature in the log register maintained in the laboratory.
- All the students should submit their records before the commencement of Laboratory experiments.
- Students should come to the lab well prepared for the experiments which are to be performed in that particular session.
- Students are asked to do the experiments on their own and should not waste their precious time by talking, roaming and sitting idle in the labs.
- Observation book and record book should be complete in all respects and it should be corrected by the staff member.
- Before leaving the laboratory, students should arrange their chairs and leave in orderly manner after completion of their scheduled time.
- Prior permission to be taken, if for some reasons, they cannot attend lab.
- Immediately report any sparks/ accidents/ injuries/ any other untoward incident to the faculty /instructor.
- In case of an emergency or accident, follow the safety procedure.
- Switch OFF the power supply after completion of experiment.

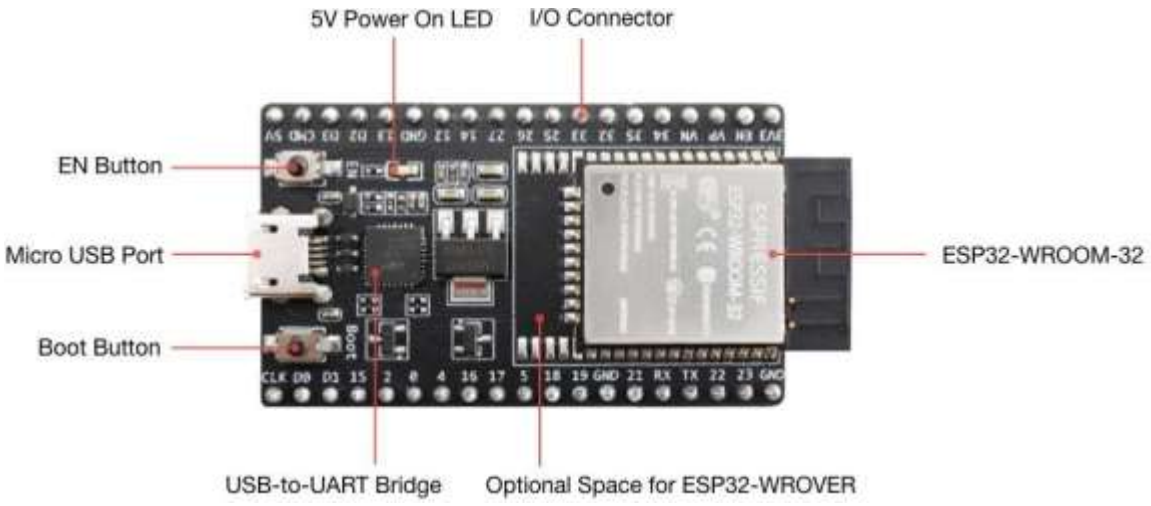
DONT's

- The use of mobile/ any other personal electronic gadgets is prohibited in the laboratory.
- Do not make noise in the Laboratory & do not sit on experiment table.
- Do not make loose connections and avoid overlapping of wires
- Don't switch on power supply without prior permission from the concerned staff.
- Never leave the experiments while in progress.
- Do not leave the Laboratory without the signature of the concerned staff in observation book.

About ESP32

ESP32 is a series of low-cost, low-power system on a chip microcontroller with integrated Wi-Fi and dual-mode Bluetooth. At the core of this module is the ESP32-D0WDQ6 chip*. The chip embedded is designed to be scalable and adaptive. There are two CPU cores that can be individually controlled, and the clock frequency is adjustable from 80 MHz to 240 MHz. The user may also power off the CPU and make use of the low-power co-processor to constantly monitor the peripherals for changes or crossing of thresholds. ESP32 integrates a rich set of peripherals, ranging from capacitive touch sensors, Hall sensors, SD card interface, Ethernet, high-speed SPI, UART, I2S and I2C.

- The ESP32 is dual core.
- It has Wi-Fi and Bluetooth built-in.
- It runs 32-bit programs.
- The clock frequency can go up to 240MHz and it has a 512 kB RAM.
- This board has 30 or 36 pins, 15 in each row.
- It also has wide variety of peripherals available, like: capacitive touch, ADCs, DACs, UART, SPI, I2C and much more.
- It comes with built-in hall effect sensor and built-in temperature sensor.



Key Component	Description
ESP32-WROOM-32	A module with ESP32 at its core.
EN	Reset button.
Boot	Download button. Holding down **Boot** and then pressing **EN** initiates Firmware Download mode for downloading firmware through the serial port.
USB-to-UART Bridge	Single USB-UART bridge chip provides transfer rates of up to 3 Mbps.

Micro USB Port	USB interface. Power supply for the board as well as the communication interface between a computer and the ESP32-WROOM-32 module.
5V Power On LED	Turns on when the USB or an external 5V power supply is connected to the board.
I/O	Most of the pins on the ESP module are broken out to the pin headers on the board. You can program ESP32 to enable multiple functions such as PWM, ADC, DAC, I2C, I2S, SPI, etc.

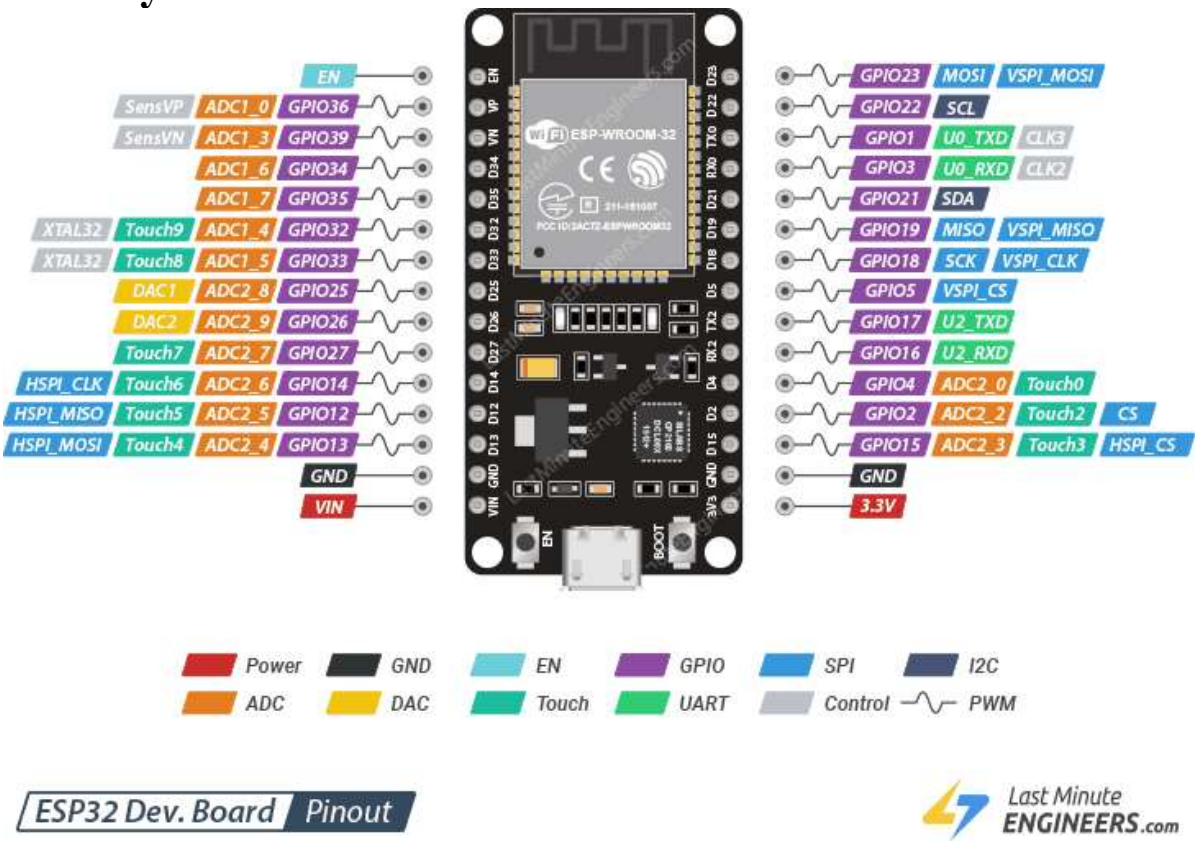
Note: The pins D0, D1, D2, D3, CMD and CLK are used internally for communication between ESP32 and SPI flash memory. They are grouped on both sides near the USB connector. Avoid using these pins, as it may disrupt access to the SPI flash memory / SPI RAM.

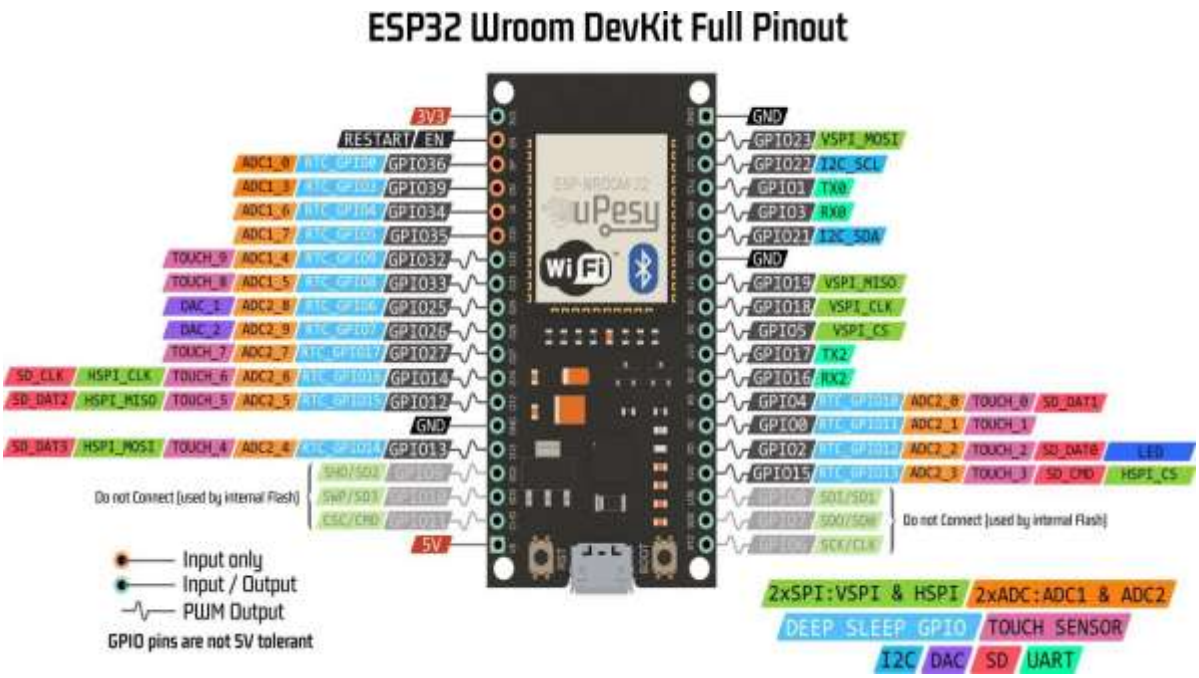
Features of the ESP32 include the following:

- **Processors:**
  - CPU: Xtensa dual-core (or single-core) 32-bit LX6 microprocessor, operating at 160 or 240 MHz and performing at up to 600 DMIPS
  - Ultra-low power (ULP) co-processor
- **Memory:** 320 KiB RAM, 448 KiB ROM
- **Wireless connectivity:**
  - Wi-Fi: 802.11 b/g/n
  - Bluetooth: v4.2 BR/EDR and BLE (shares the radio with Wi-Fi)
- **Peripheral interfaces:**
  - 34 × programmable GPIOs
  - 12-bit SAR ADC up to 18 channels
  - 2 × 8-bit DACs
  - 10 × touch sensors (capacitive sensing GPIOs)
  - 4 × SPI
  - 2 × I²S interfaces
  - 2 × I²C interfaces
  - 3 × UART
  - SD/SDIO/CE-ATA/MMC/eMMC host controller
  - SDIO/SPI slave controller
  - Ethernet MAC interface with dedicated DMA and planned IEEE 1588 Precision Time Protocol support[4]
  - CAN bus 2.0
  - Infrared remote controller (TX/RX, up to 8 channels)
  - Motor PWM
  - LED PWM (up to 16 channels)
  - Hall effect sensor
  - Ultra-low power analog pre-amplifier

- **Security:**
  - IEEE 802.11 standard security features all supported, including WPA, WPA2, WPA3 (depending on version)[5] and WLAN Authentication and Privacy Infrastructure (WAPI)
  - Secure boot
  - Flash encryption
  - 1024-bit OTP, up to 768-bit for customers
  - Cryptographic hardware acceleration: AES, SHA-2, RSA, elliptic curve cryptography (ECC), random number generator (RNG)
- **Power management:**
  - Internal low-dropout regulator
  - Individual power domain for RTC
  - 5 µA deep sleep current
  - Wake up from GPIO interrupt, timer, ADC measurements, capacitive touch sensor interrupt

Pin Layout:





For more details : [https://www.upesy.com/blogs/tutorials/esp32-pinout-reference-gpio-pins- ultimate-guide](https://www.upesy.com/blogs/tutorials/esp32-pinout-reference-gpio-pins-ultimate-guide)
[https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32d_esp32- wroom-32u_datasheet_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32d_esp32-wroom-32u_datasheet_en.pdf)

Install the Arduino Desktop IDE:

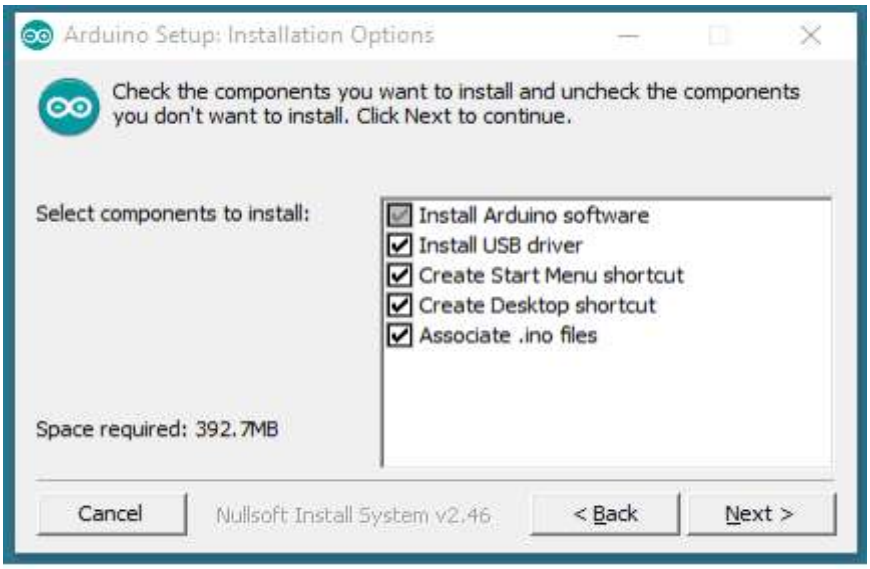
Arduino IDE Installation (Windows):

Step 1: Download the Arduino Software (IDE):

https://www.arduino.cc/en/software

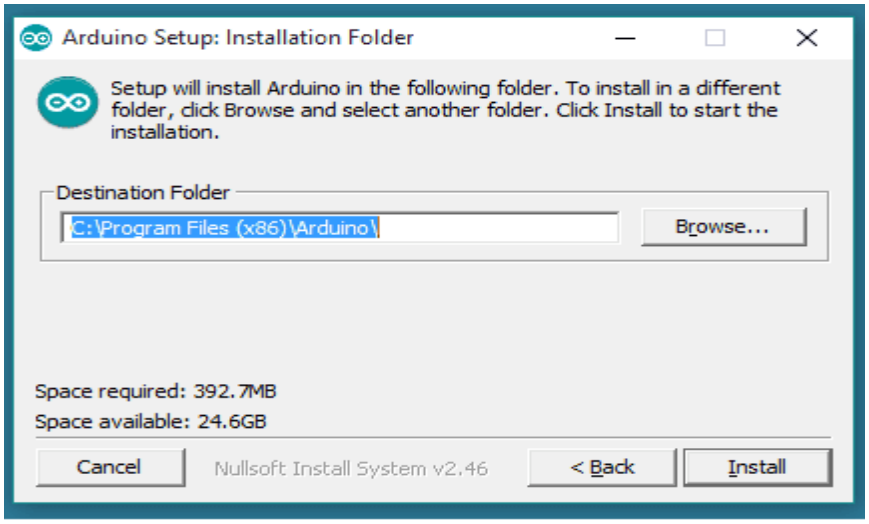
Step 2:

When the download finishes, proceed with the installation and please allow the driver installation process when you get a warning from the operating system.



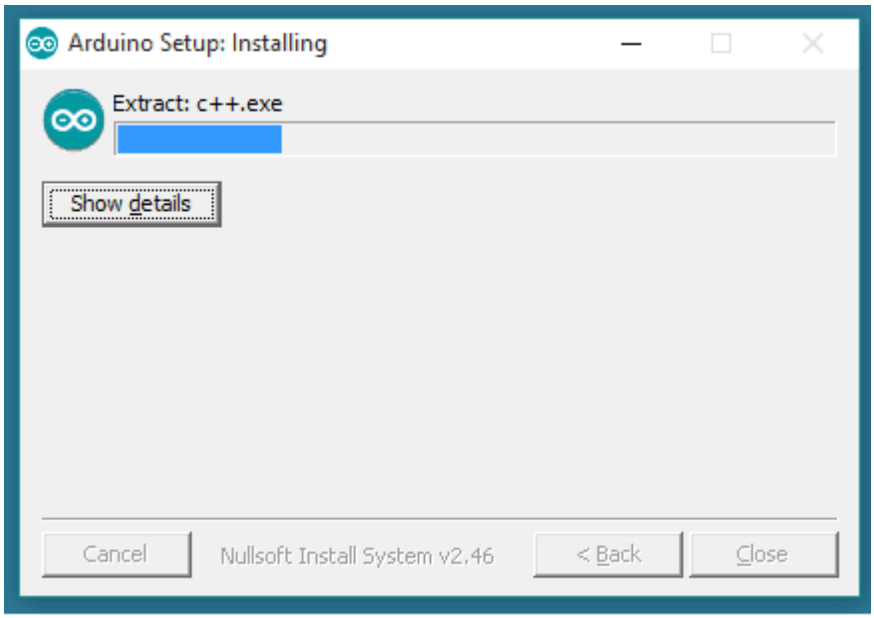
Choose the components to install.

Step 3:



Choose the installation directory.

Step 4:



Installation in progress.

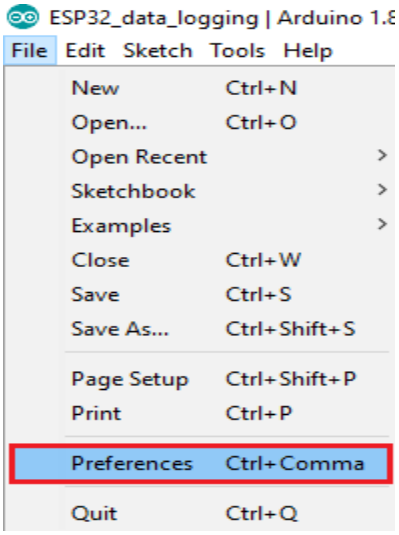
The process will extract and install all the required files to execute properly the Arduino Software (IDE)

Installing the ESP32 Board in Arduino IDE (Windows, Mac OS X, Linux)

There's an add-on for the Arduino IDE that allows you to program the ESP32 using the Arduino IDE and its programming language.

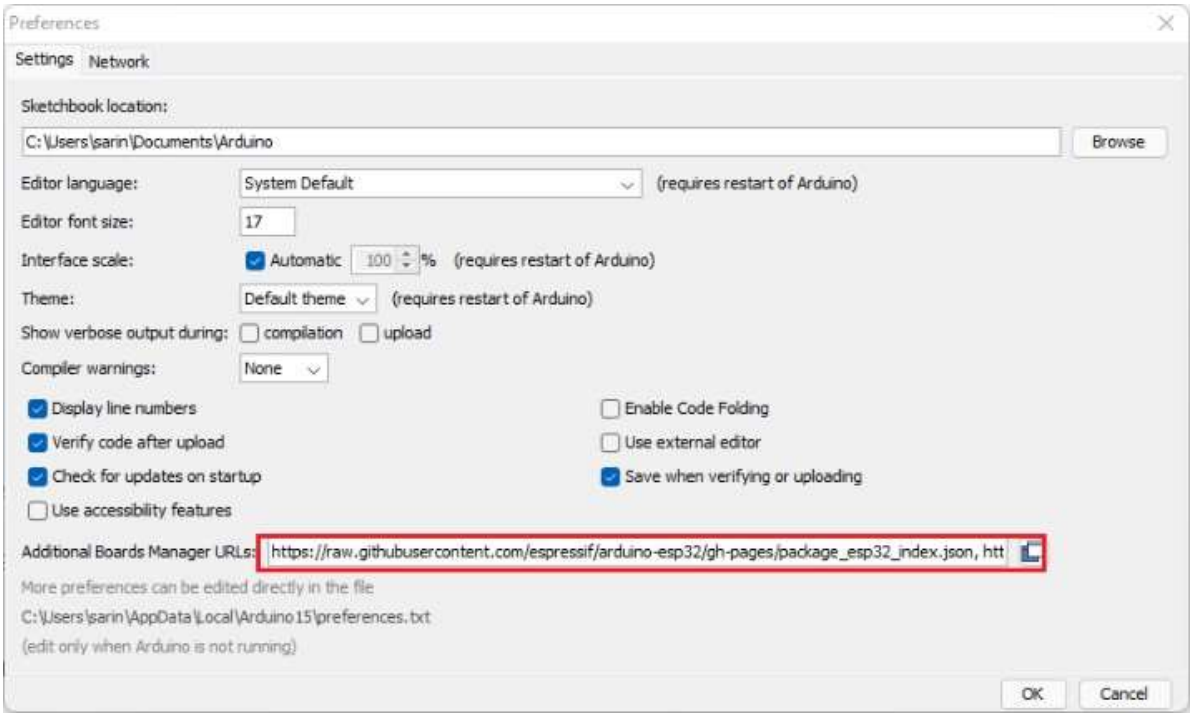
To install the ESP32 board in your Arduino IDE, follow these Steps:

Step 1: In your Arduino IDE, go to File> Preferences



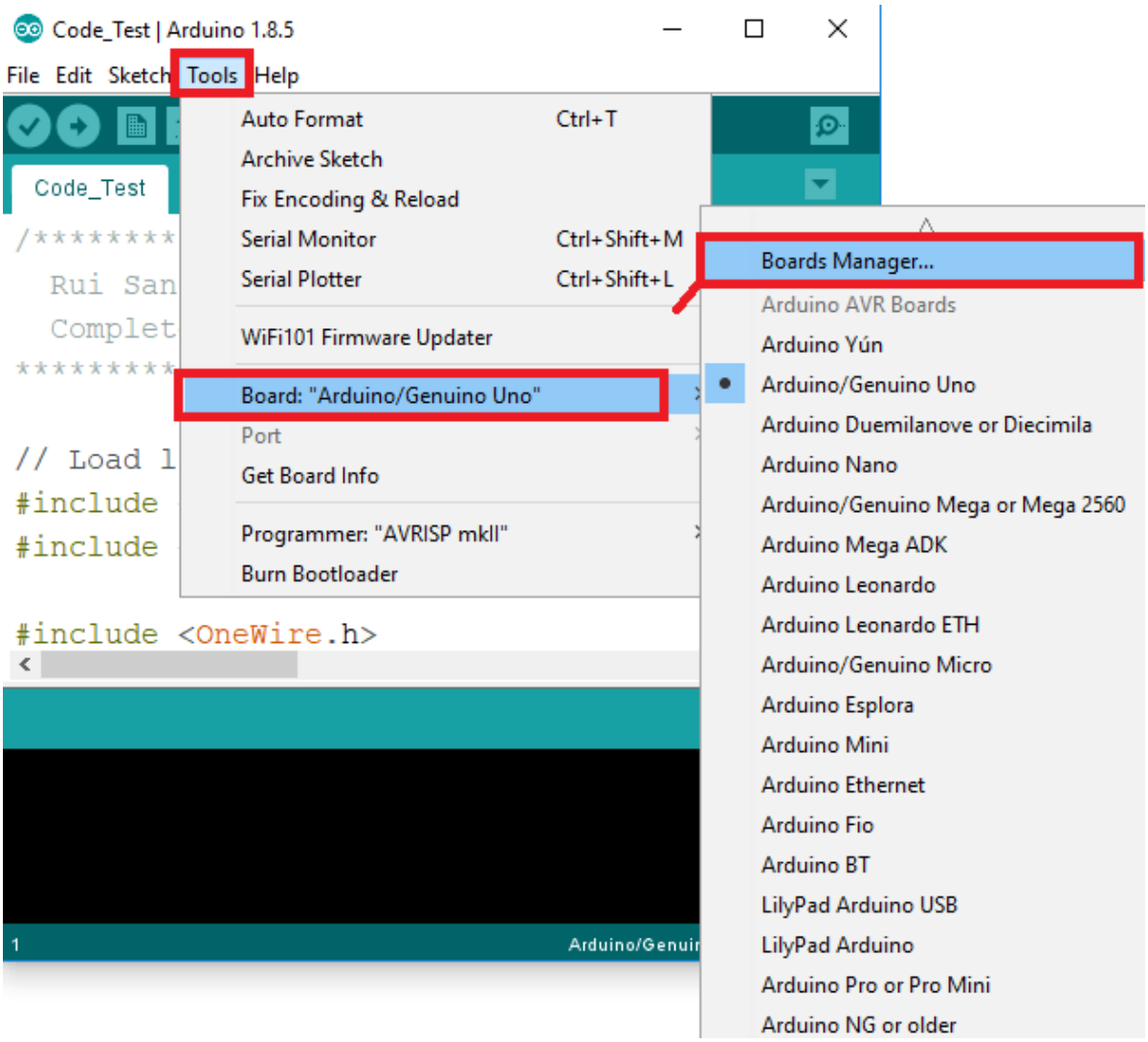
Step 2: Enter the following into the "Additional Board Manager URLs" field:
`https://raw.githubusercontent.com/espressif/arduino-esp32/gh- pages/package_esp32_index.json`

Then, click the "OK" button:

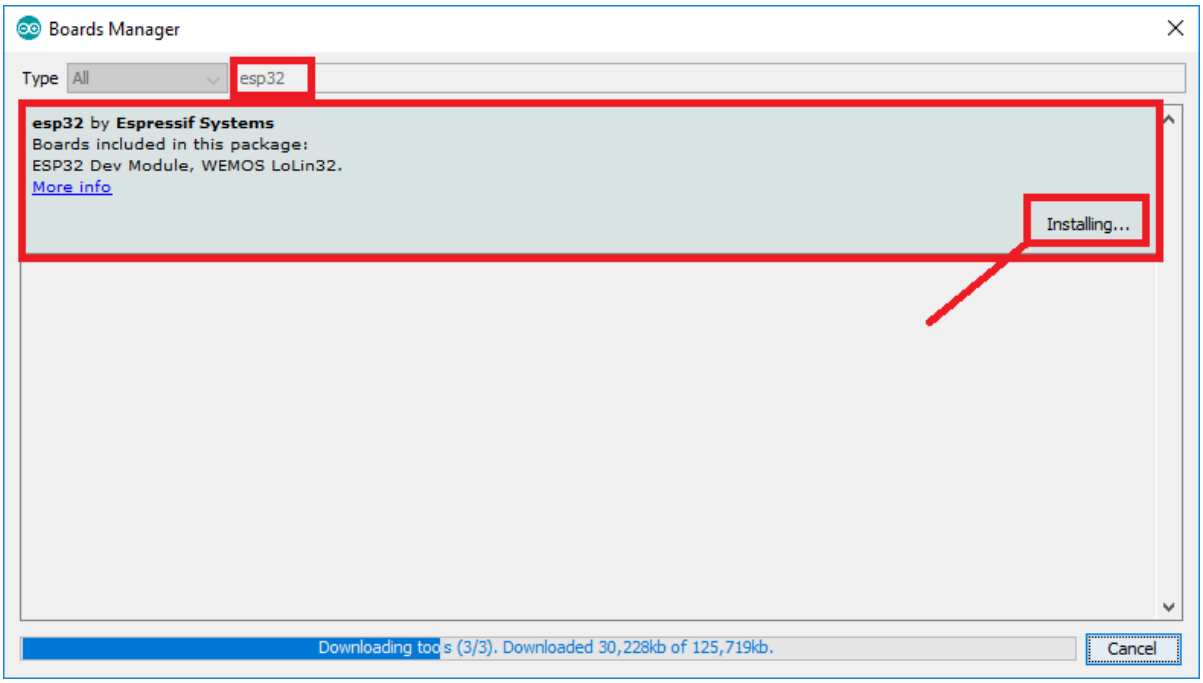


Note: if you already have the ESP8266 boards URL, you can separate the URLs with a comma as follows:
`https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json,`
`http://arduino.esp8266.com/stable/package_esp8266com_index.json`

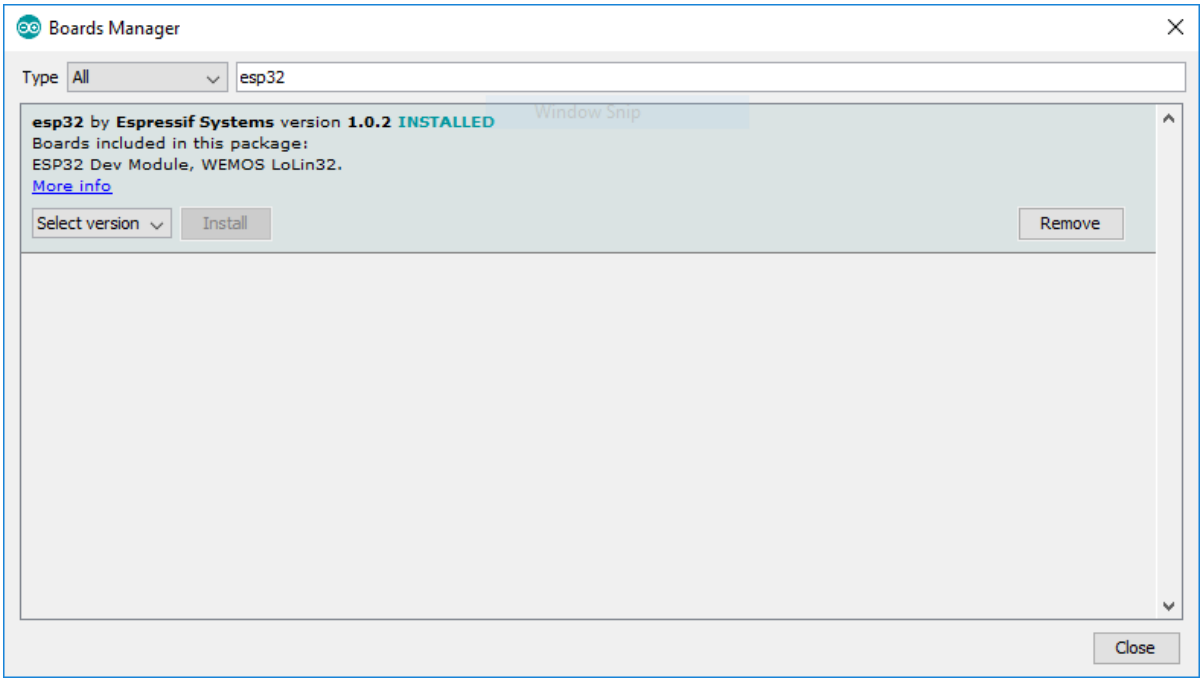
Step 3: Open the Boards Manager. **Go to Tools > Board > Boards Manager…**



Step 4: Search for ESP32 and press install button for the "**ESP32 by Espressif Systems**":



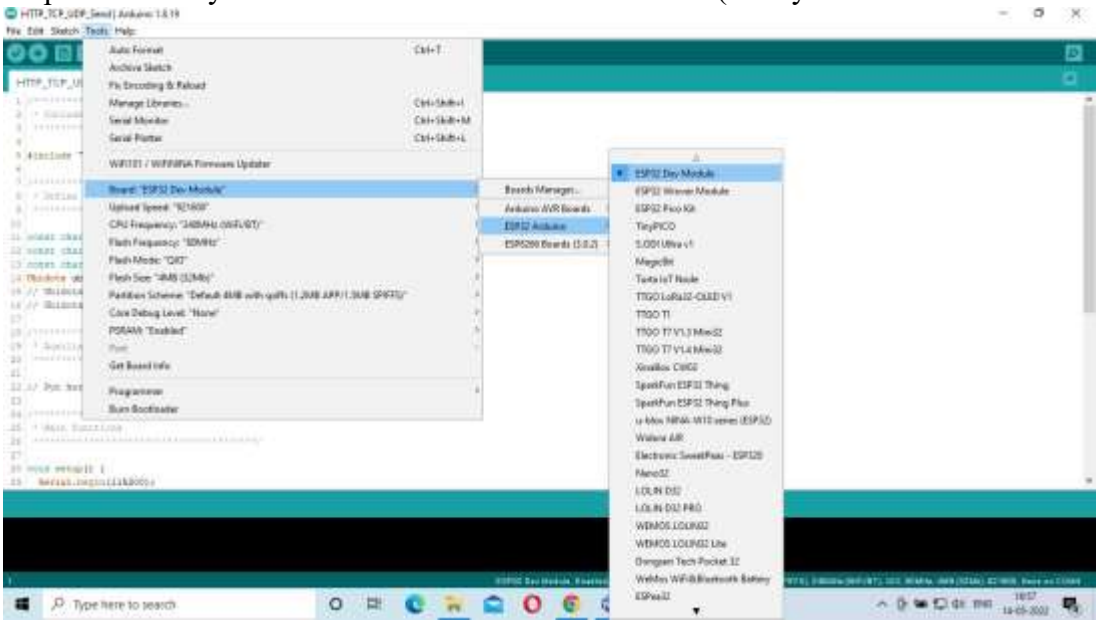
Step 5: That's it. It should be installed after a few seconds.



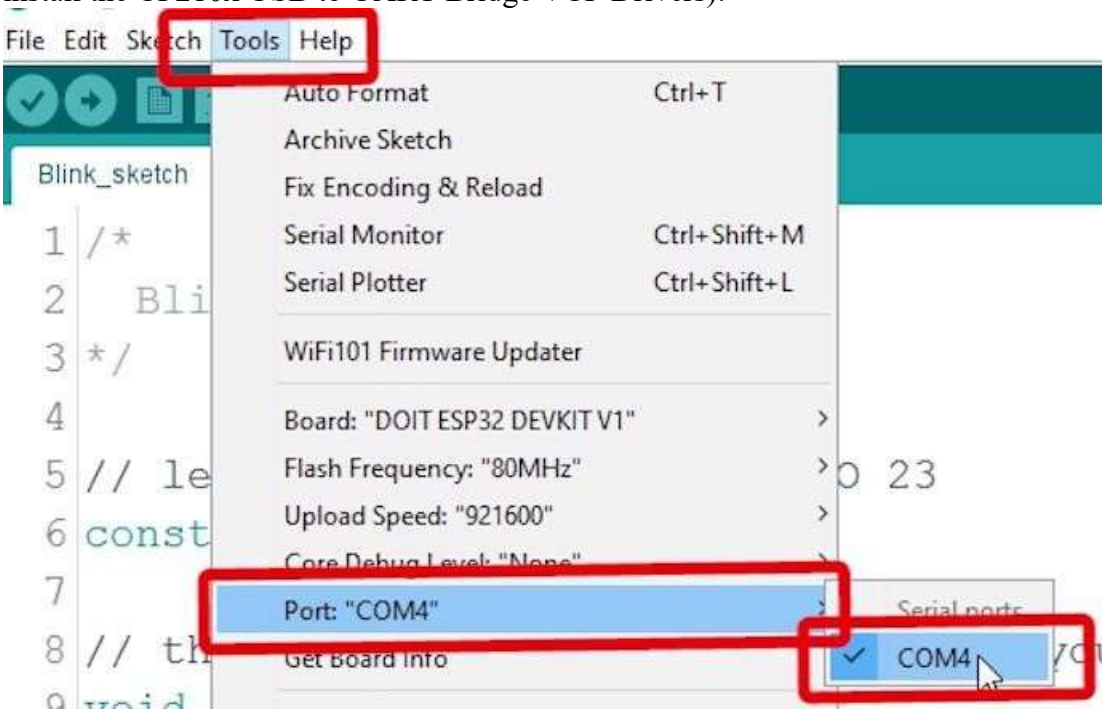
Testing the Installation:

Plug the ESP32 board to your computer. With your Arduino IDE open, follow these steps:

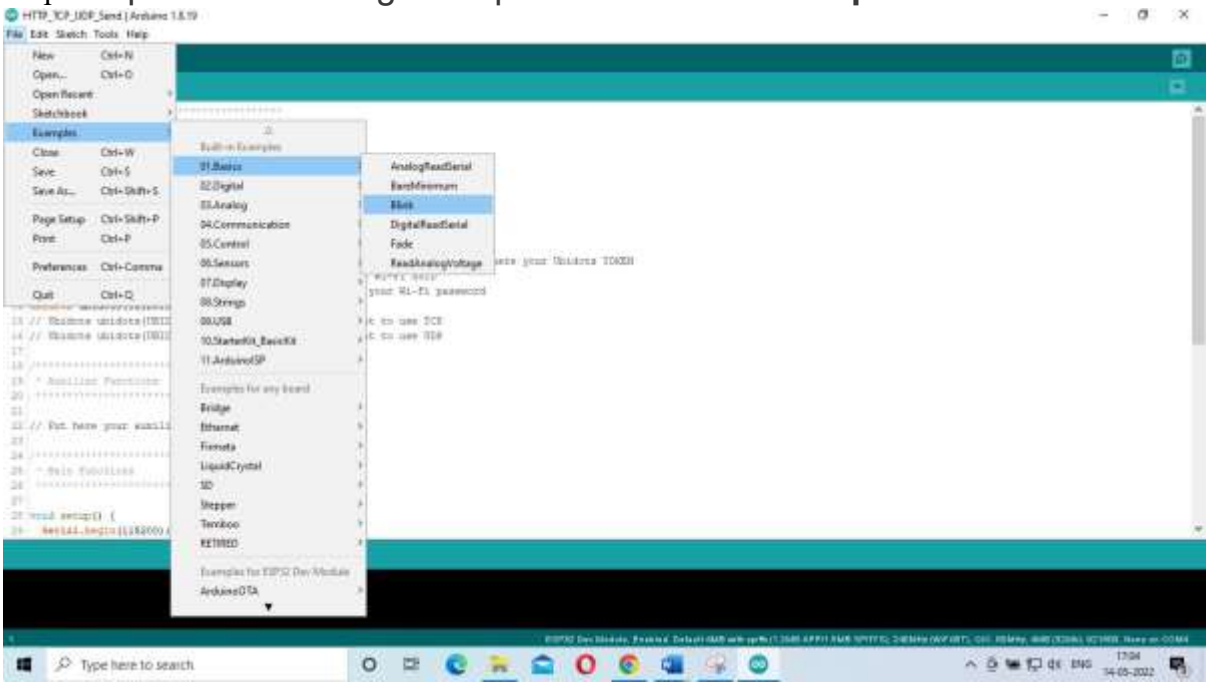
Step 1: Select your Board in Tools > Board menu (in my case it's the "ESP32 Dev Module")



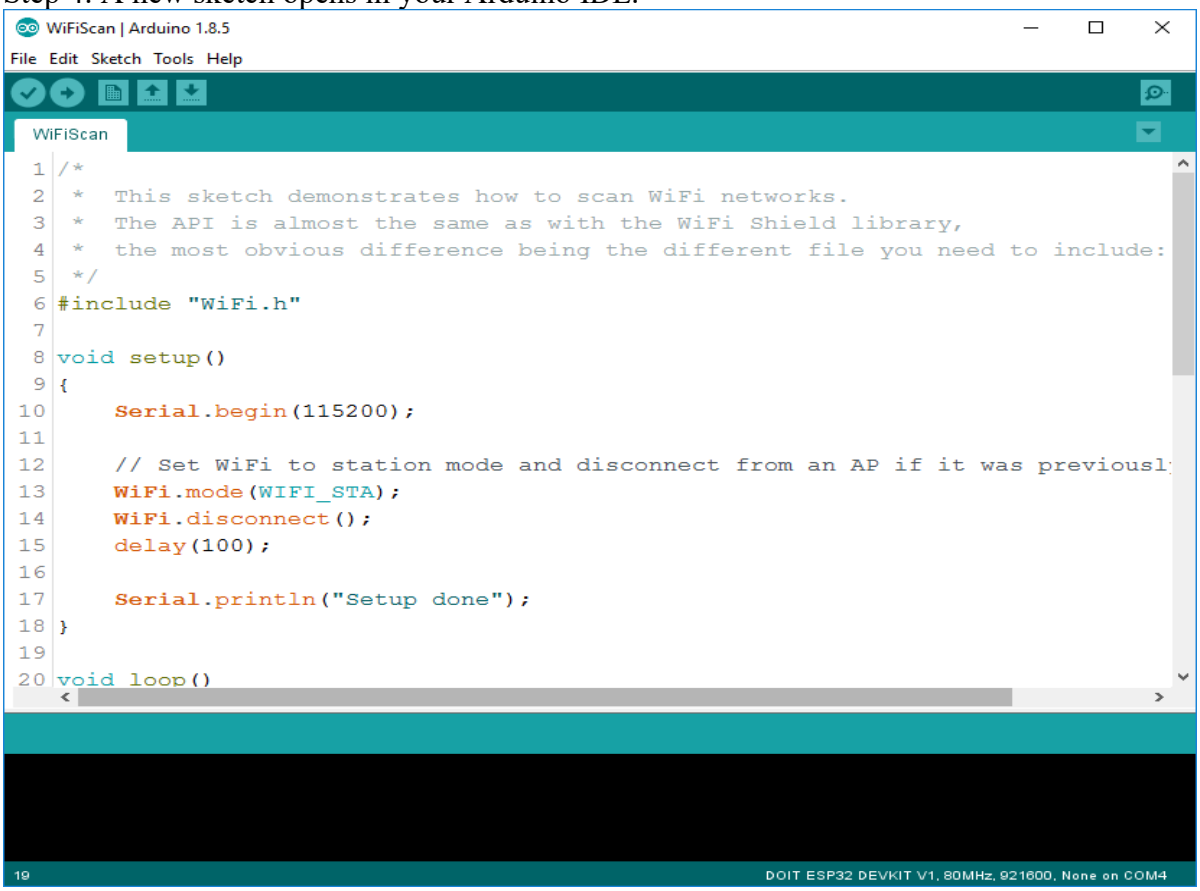
Step 2: Select the Port (if you don't see the COM Port in your Arduino IDE, you need to install the CP210x USB to UART Bridge VCP Drivers):



Step 3: Open the following example under **File > Examples >Basics> Blink**

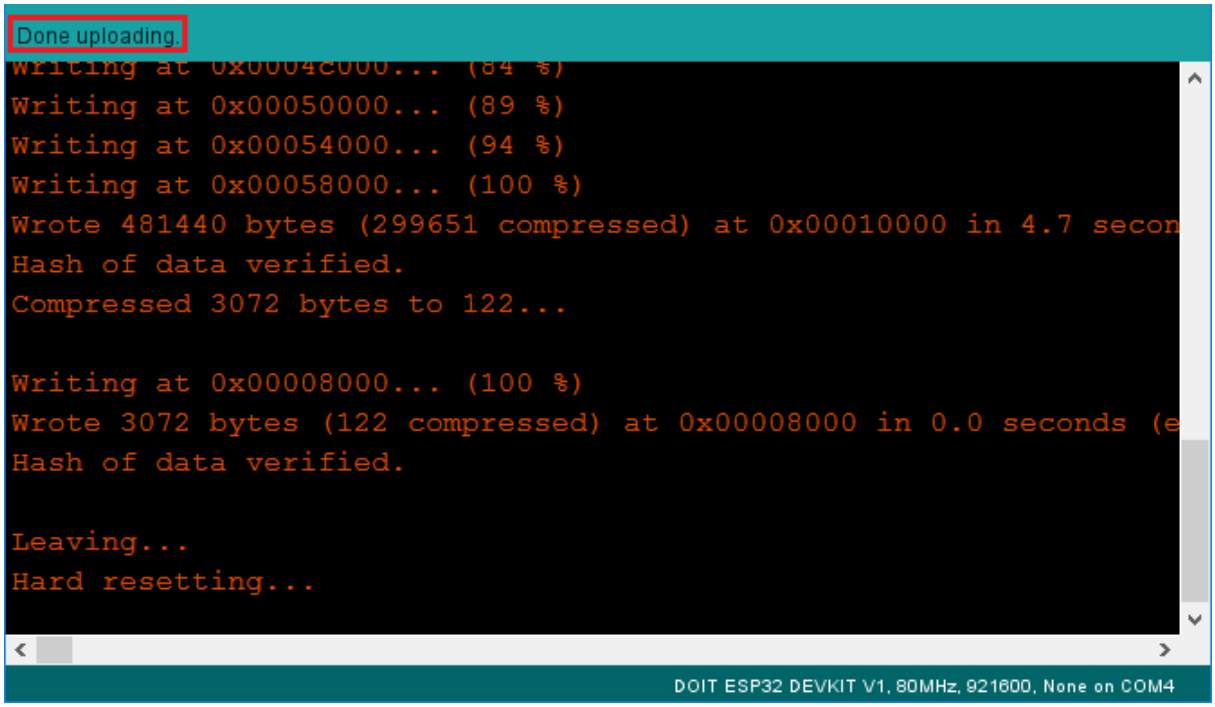


Step 4: A new sketch opens in your Arduino IDE:



Step 5: Press the Upload button in the Arduino IDE. Wait a few seconds while the code compiles and uploads to your board.

Step 6: If everything went as expected, you should see a “Done uploading.” message.



More details : https://randomnerdtutorials.com/installing-the-esp32-board-in-arduino-ide- windows-instructions/

EXPERIMENT NO. 1 a) Date:

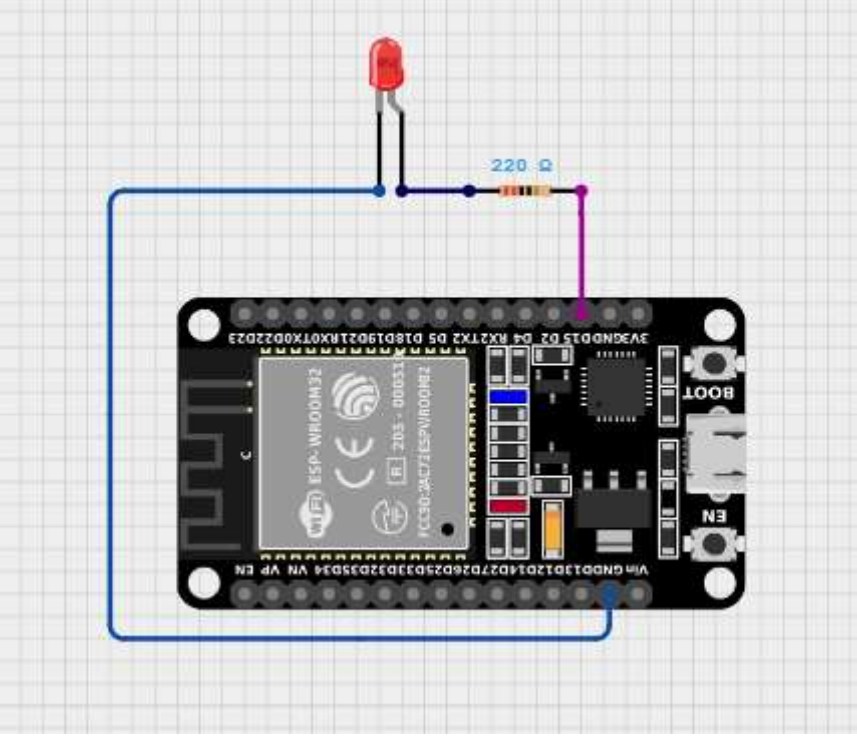
Aim: Design an interface circuit to control LED light using ESP32.

Description: Connect an LED to an ESP32 and create a program that switches the LED on for 1 and OFF every 2 seconds.

Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	LED	1
3	Micro USB cable	1

Connection Diagram:



Procedure:

1. Plug the ESP32 development board to your PC
2. Make the connection as per the connection diagram
3. Open the Arduino IDE in computer and write the program. Save the new sketch in your working directory

Note: **Make sure you have the right board and COM port selected in your Arduino IDE settings**.

4. Compile the program and upload it to the ESP32 Development Board. If everything went as expected, you should see a "Done uploading" message. (You need to hold the ESP32 on-board Boot button while uploading).

Code:

```
//external LED

#define LED_PIN 15

//pin number 2 for Internal LED

void setup() {

pinMode (LED_PIN, OUTPUT);

}
void loop() {

// Blink LED every 1 second
digitalWrite(LED_PIN, HIGH);

delay(1000);
digitalWrite(LED_PIN, LOW);
delay(2000);
}
```

Assignment–1 (Application-based Scenario):

In a smart ambient lighting system, lighting intensity is adjusted gradually to improve visual comfort and save power. Modify the given LED blinking program so that the LED smoothly fades from OFF to maximum brightness and back to OFF, instead of abrupt ON–OFF switching.

Code and Output:

Assignment–2 (Simulation Task):

Simulate the original LED blinking program using an online simulation tool such as TinkerCAD. Verify the timing behaviour of the LED and capture the simulation output.

LED working: https://youtu.be/O8M2z2hIbag?si=J82TJVVnlQ5xsFq1

Applications:

-
-
-
-
-

Work Sheet

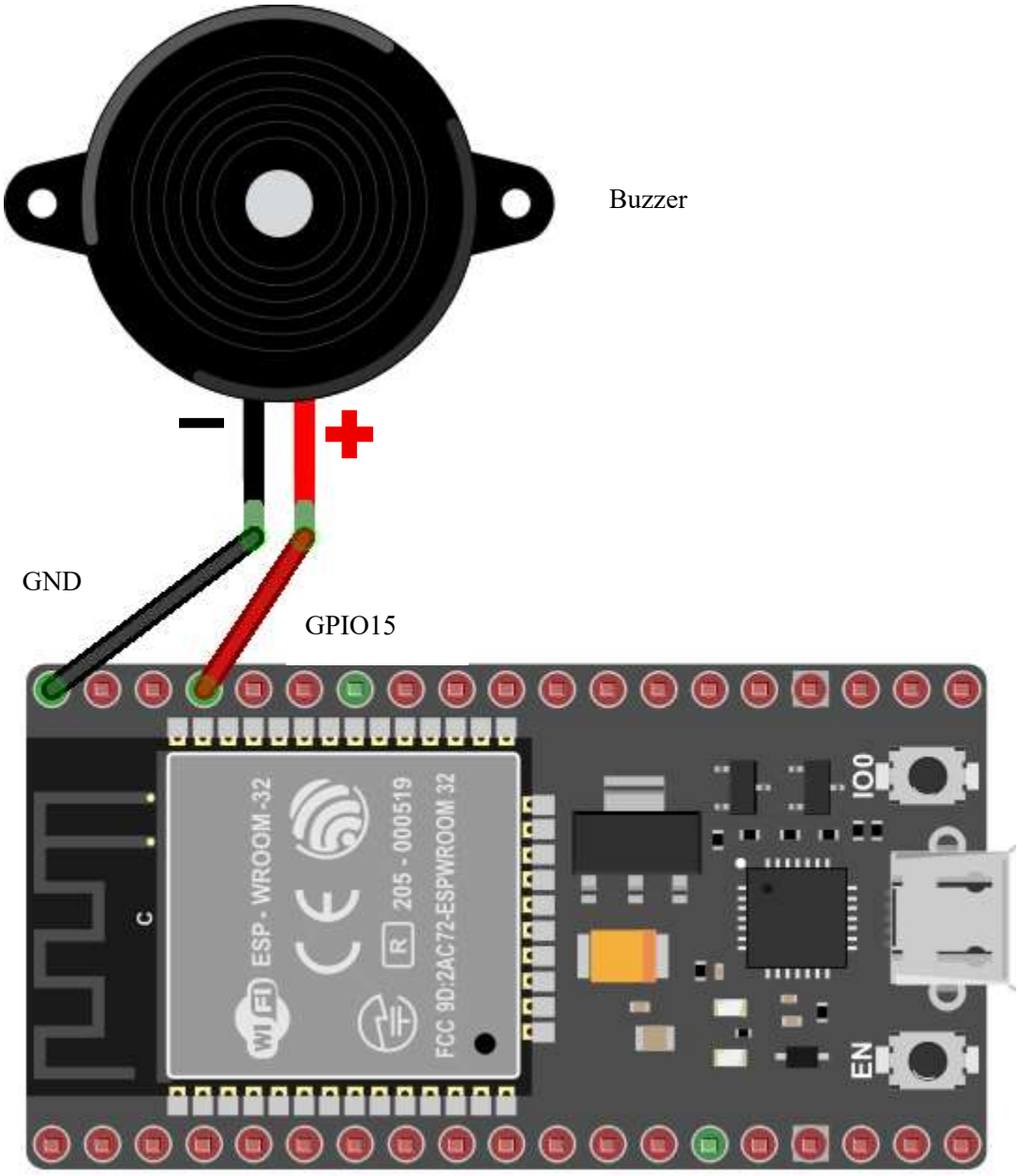
EXPERIMENT NO. 1 b) Date:

Aim: Design an interface circuit to control Buzzer using ESP32

Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	Buzzer	1
3	Micro USB cable	1

Connection Diagram:



Description: We often encounter buzzers in common devices such as alarm clocks, doorbells ,or other household appliances. The main function of this device is to convert an electrical signal to audio sound.
https://www.elprocus.com/buzzer-working-applications/

Code:

```
#define buzzerPin 15  // buzzer Pin (+) is connected GPIO15
void setup() {
  // put your setup code here, to run once:
 pinMode(buzzerPin, OUTPUT); // buzzer IS OUTPUT DEVICE
}

void loop() {
  // put your main code here, to run repeatedly:

  for(int i = 0; i< 50 ; i++)  // Loop 50 times and play a short tone
  {
    digitalWrite(buzzerPin, HIGH); / High to turn ON the buzzer
    delay(3); // Wait for 3ms
    digitalWrite(buzzerPin, LOW); // Low to turn OFF the buzzer
    delay(3); // Wait for 3ms
    }
  delay(1000); // Wait for 1s before starting the next loop
}
```

Signature of Staff In-charge with date

EXPERIMENT NO. 2 Date:

<u>Aim:</u> Design an Interface circuit for DHT11 (Temperature and Humidity) Sensor with ESP32. Write a Program to display Temperature and Humidity on Serial Monitor.

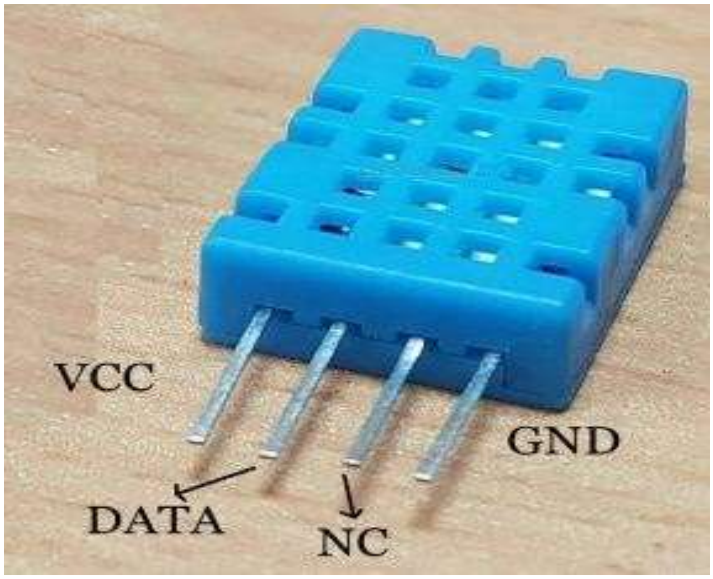
Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	DHT11 Digital Temperature and Humidity sensor	1
3	Jumper wires	3
4	Micro USB cable	1

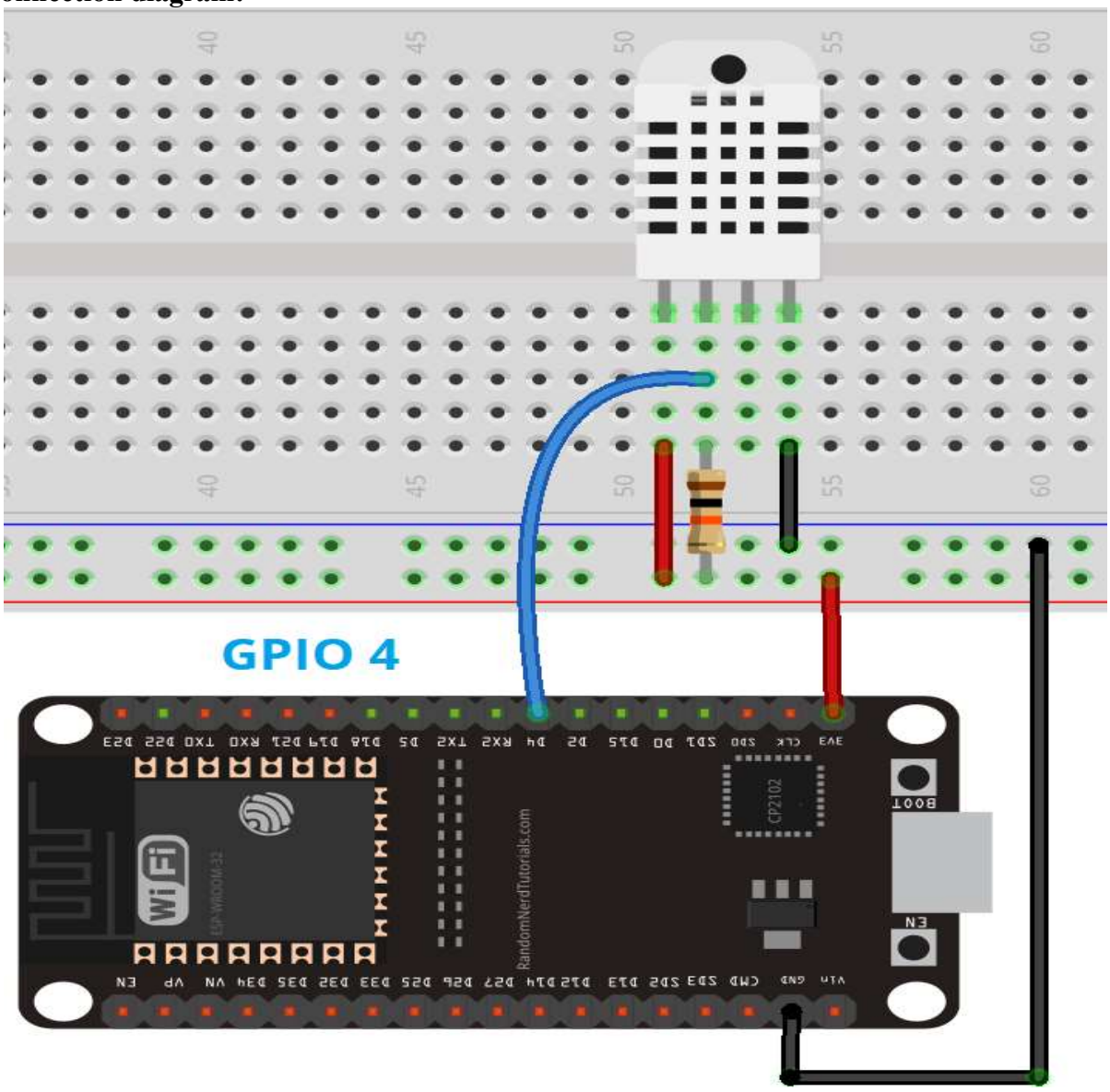
The DHT11 sensors are used to measure temperature and relative humidity.

- Temperature range: 0 to 50ºC +/- 2ºC
- Relative humidity range: 20 to 90% +/-5%
- Temperature resolution: 1ºC
- Humidity resolution: 1%
- Operating voltage: 3 to 5.5 V DC
- Current supply: 0.5 to 2.5 mA
- Sampling period: 1second

DHT11 sensor:



Connection diagram:



Procedure:

1. Plug the ESP32 development board to your PC.
2. Make the connection as per the connection diagram.
3. Open the Arduino IDE in computer and write the program. Save the new sketch in your working directory.

Note: **Make sure you have the right board and COM port selected in your Arduino IDE settings**.

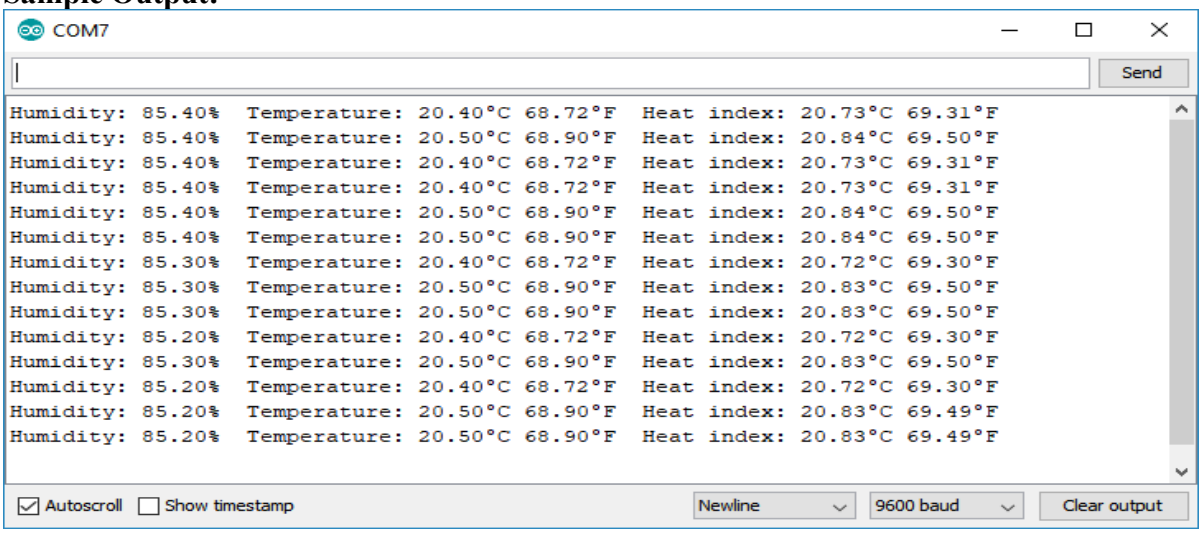
4. Compile the program and upload it to the ESP32 Development Board. If everything went as expected, you should see a "Done uploading" message. (You need to hold the ESP32 on-board Boot button while uploading).
5. After uploading the code, open the Serial Monitor at a baud rate of 115200.

Code:

```
#include "DHT.h"

#define DHTPIN 4     // Digital pin connected to the DHT sensor
// Feather HUZZAH ESP8266 note: use pins 3, 4, 5, 12, 13 or 14 --
// Pin 15 can work but DHT must be disconnected during program upload.

// Uncomment whatever type you're using!
#define DHTTYPE DHT11   // DHT 11

//#define DHTTYPE DHT22   // DHT 22  (AM2302), AM2321
//#define DHTTYPE DHT21   // DHT 21 (AM2301)

// Connect pin 1 (on the left) of the sensor to +5V
// NOTE: If using a board with 3.3V logic like an Arduino Due connect pin 1
// to 3.3V instead of 5V!
// Connect pin 2 of the sensor to whatever your DHTPIN is
// Connect pin 4 (on the right) of the sensor to GROUND
// Connect a 10K resistor from pin 2 (data) to pin 1 (power) of the sensor

// Initialize DHT sensor.
// Note that older versions of this library took an optional third parameter to
// tweak the timings for faster processors.  This parameter is no longer needed
// as the current DHT reading algorithm adjusts itself to work on faster procs.
DHT dht(DHTPIN, DHTTYPE);

void setup() { Serial.begin(115200);
  Serial.println(F("DHTxx test!"));

  dht.begin();
}

void loop() {
  // Wait a few seconds between measurements.
  delay(2000);

  // Reading temperature or humidity takes about 250 milliseconds!
  // Sensor readings may also be up to 2 seconds 'old' (its a very slow sensor)
  float h = dht.readHumidity();
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();
  // Read temperature as Fahrenheit (isFahrenheit = true)
   float f = dht.readTemperature(true);

  // Check if any reads failed and exit early (to try again).
```

```
  if (isnan(h) || isnan(t) || isnan(f)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }

  // Compute heat index in Fahrenheit (the default)
  float hif = dht.computeHeatIndex(f, h);
  // Compute heat index in Celsius (isFahreheit = false)
  float hic = dht.computeHeatIndex(t, h, false);

  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
  Serial.print(t);
  Serial.print(F("\xC2\xB0 C, "));
  Serial.print(f);
  Serial.print(F("\xC2\xB0 F,  Heat index: "));
  Serial.print(hic);
  Serial.print(F("\xC2\xB0 C, "));
  Serial.print(hif);
  Serial.println(F("\xC2\xB0 F."));
}
```

Sample Output:



Assignment–1 (Application-based Scenario):

In weather monitoring and HVAC systems, dew point temperature is a critical parameter to predict condensation and human comfort. Modify the existing program to calculate and display the Dew Point Temperature using the Magnus formula based on the measured temperature and humidity values.

Code and Output:

Assignment–2 (Simulation Task):

Simulate the existing DHT sensor program in TinkerCAD by varying temperature and humidity values. Observe and record the corresponding changes in sensor readings. Capture the simulation output.

DHT11 sensor working: https://youtu.be/p_aKKp_nRUM?si=jrUJqVIaAKCaTlhu

Applications:

-
-
-
-
-

Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 3 Date:

Aim: Design an interface circuit with 16x2 I2C LCD AND ESP32 to display characters on LCD.

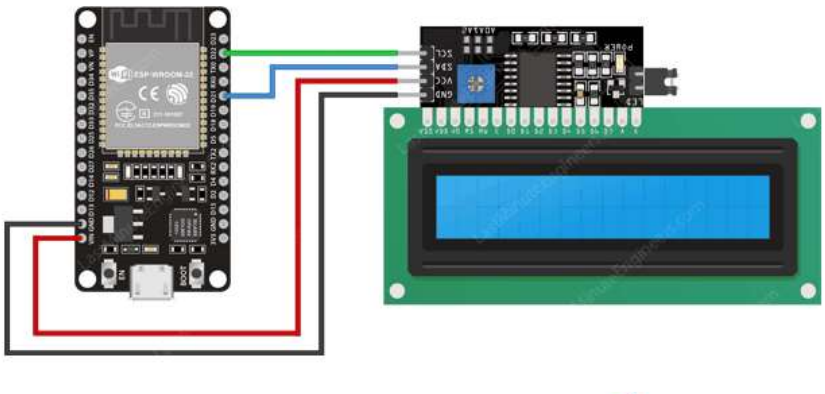
Description: Character LCDs are specially designed to display letters, numbers, and symbols. A 16×2 character LCD, for example, can show 16 characters across each line, with two lines total.

I2C LCD Adapter

The key component of this adapter is an 8-bit I/O expander chip called PCF8574. This chip converts the I2C data from your ESP32 into the parallel data that the LCD display needs to function.

https://lastminuteengineers.com/esp32-i2c-lcd-tutorial/

Connection Diagram:



Determining the I2C Address

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);

  //  wait for serial port to connect
  while (!Serial) {
  }

  Serial.println();
  Serial.println("I2C scanner. Scanning ...");
  byte count = 0;

  Wire.begin();
  for (byte i = 8; i < 120; i++) {
    Wire.beginTransmission(i);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found address: ");
      Serial.print(i, DEC);
      Serial.print(" (0x");
      Serial.print(i, HEX);
      Serial.println(")");
      count++;
      delay(1);  // maybe unneeded?
    }           // end of good response
  }             // end of for loop
  Serial.println("Done.");
  Serial.print("Found ");
  Serial.print(count, DEC);
  Serial.println(" device(s).");
}  // end of setup

void loop() {
}
```

Code -1 : Display static text on LCD

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x3F, 16, 2);  // set the LCD address to 0x3F for a 16 chars and 2 line display

void setup() {
  lcd.init();
  lcd.clear();
  lcd.backlight();  // Make sure backlight is on

  // Print a message on both lines of the LCD.
  lcd.setCursor(2, 0);  //Set cursor to character 2 on line 0
  lcd.print("Hello world!");

  lcd.setCursor(2, 1);  //Move cursor to character 2 on line 1
  lcd.print("LCD data");
}

void loop()
{

}
```

Output:

Code -2: Scrolling Text

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x3F,16,2);  // set the LCD address to 0x3F for a 16 chars and 2 line display

void setup() {
  lcd.init();
  lcd.clear();
  lcd.backlight();      // Make sure backlight is on

  // Print a message
  lcd.print("Scrolling Text Demo");
  delay(1000);  // pause to read the message initially
}

void loop() {
  lcd.scrollDisplayLeft();   // scroll everything to the left by one position
```

```
  delay(300);              // small delay for visible scrolling speed
}
```

Sample output:

Assignment:

Analyse the given code. What does the below program output when uploaded ? Capture the output.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Set the LCD address to 0x27 or 0x3F for a 16 chars and 2 line display
LiquidCrystal_I2C lcd(0x27, 16, 2);
// Define custom character
byte Bell[8] = {
  0b00100,
  0b01110,
  0b01110,
  0b01110,
  0b11111,
  0b00000,
  0b00100,
  0b00000
};
```

```
void setup() {
  lcd.init();
  lcd.backlight();

  // Create the custom character at location 0
  lcd.createChar(0, heart);

  lcd.setCursor(0, 0);
  lcd.print("Custom Char:");

  // Display the custom character
  lcd.setCursor(0, 1);
  lcd.write(byte(0));
}

void loop() {
}
```

LCD WORKING: https://youtu.be/VamqtyatBss?si=LB1UcYHZ75uaZidF

Applications:

-
-
-
-
-

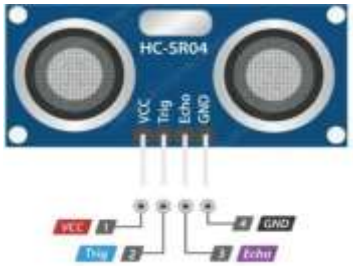
Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 4 Date:

Aim: Design an interface circuit for Ultrasonic Sensor (HC-SR04) with ESP32. Write a program to measure the distance and display on Serial Monitor.

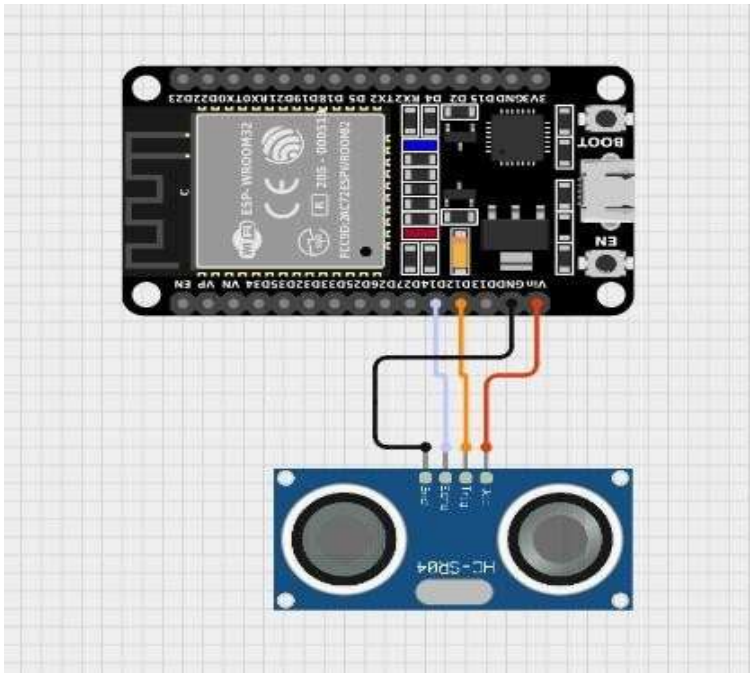
Ultrasonic sensor:



The HC-SR04 Ultrasonic Range Sensor Features:

- Input Voltage: 5V
- Current Draw: 20mA (Max)
- Digital Output: 5V
- Digital Output: 0V (Low)
- Working Temperature: -15°C to 70°C
- Sensing Angle: 30° Cone
- Angle of Effect: 15° Cone
- Ultrasonic Frequency: 40kHz
- Range: 2cm - 400cm. The HC-SR04 sensor works best between 2cm – 400 cm (1" - 13ft) within a 30 degree cone, and is accurate to the nearest 0.3cm

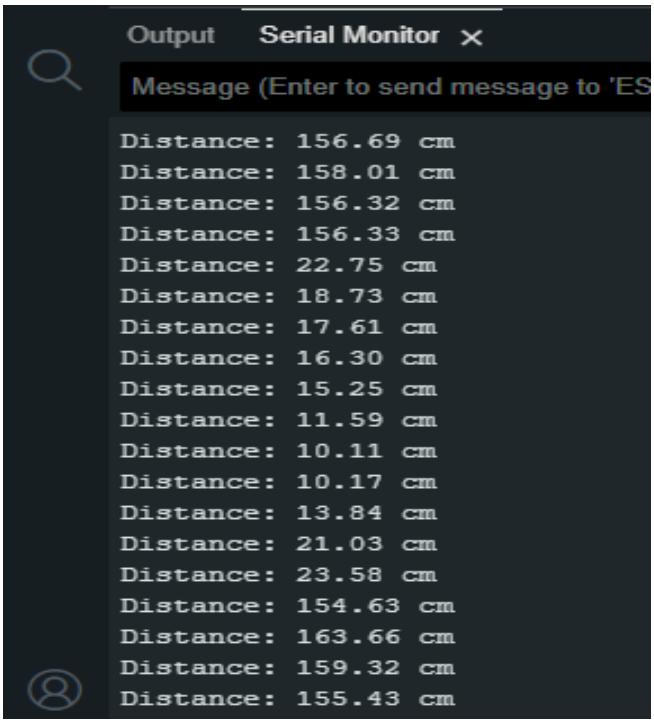
Connection diagram:



Code:

```
#define TRIG_PIN 12
#define ECHO_PIN 14
void setup() {
Serial.begin(9600);
pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);
}
void loop() {
digitalWrite(TRIG_PIN, LOW);
delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH);
delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);
long duration = pulseIn(ECHO_PIN, HIGH);
float distance = duration * 0.034 / 2;
Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");
delay(500);
}
```

Sample Output on serial monitor:



Assignment–*1 (Application-based Scenario):*

In smart water management systems, ultrasonic sensors are widely used to monitor water levels in overhead tanks. Modify the given program to indicate water level status (Low / Medium / High) based on the distance between the sensor and the water surface.

Code and Output:

Assignment–2 (Simulation Task):

Simulate the existing ultrasonic distance measurement code using TinkerCAD by varying object distance and observe the corresponding distance values. Capture the simulation output.

Ultrasonic sensor working: https://youtu.be/iwD9RiqIHQw?si=kdhHqMJuaXeL2VI8

Applications:

-
-
-
-
-

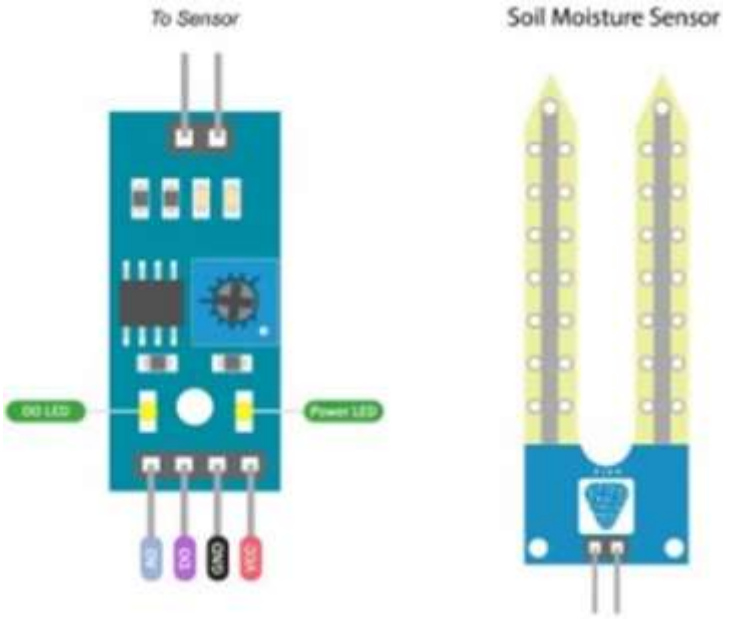
Work Sheet

Signature of Staff In-charge with date

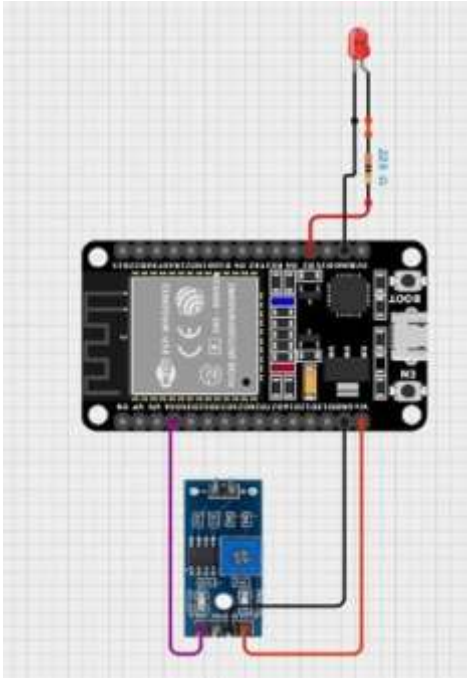
EXPERIMENT NO. 5 Date:

Aim: Design an interface circuit for Soil Moisture sensor with ESP32 to measure soil moisture and display it on a Serial monitor.

Soil moisture sensor:



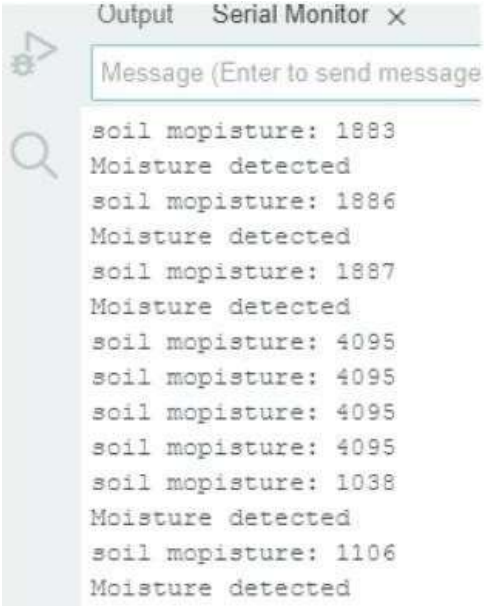
Connection diagram:



Code:

```
#define soil_PIN 34
#define LED_PIN 15
int THRESHOLD=2500;
void setup() {
pinMode(LED_PIN, OUTPUT);
Serial.begin(115200);
}

void loop() {
int soilValue = analogRead(soil_PIN);
Serial.print("soil moisture: ");
Serial.println(soilValue);

if (soilValue < THRESHOLD)
{
digitalWrite(LED_PIN, HIGH);
Serial.println("Moisture detected");
}
else
{
digitalWrite(LED_PIN, LOW);
}
delay(1000);
}
```

Sample Output on serial monitor:



Assignment–1 (Application-based Scenario):

To support Swachh Bharath, smart waste management systems are used to automatically segregate waste and improve recycling efficiency. Modify the program to classify waste as Dry or Wet based on moisture sensor readings using suitable threshold values.

Code and Output:

Assignment–2 (Simulation Task):

Simulate the existing moisture sensor interfacing program in TinkerCAD by varying sensor values and observe the Dry/Wet indication logic. Capture the simulation output.

Resistance-based Soil Moisture Sensor working: https://lastminuteengineers.com/soil-moisture-sensor-arduino-tutorial/

Applications:

-
-
-
-
-

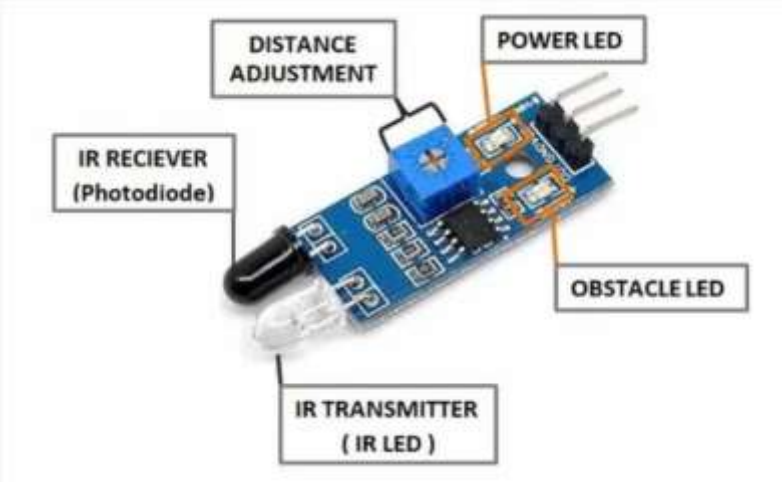
Work Sheet

Signature of Staff In-charge with date

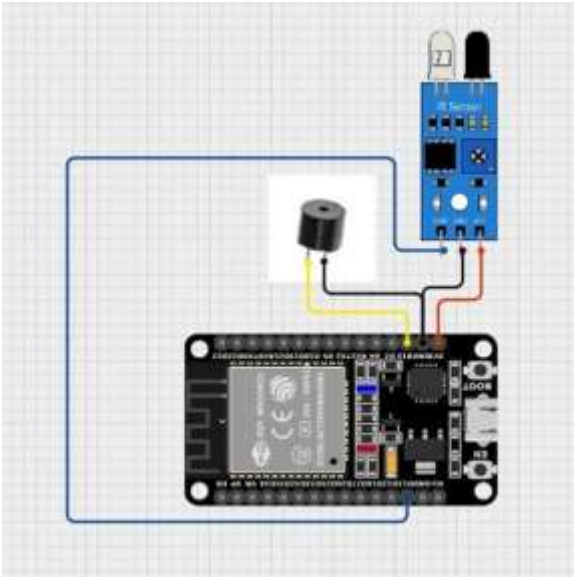
EXPERIMENT NO. 6 Date:

Aim: Design an interface circuit to detect the motion using IR sensor and turn on Buzzer upon detection.

IR sensor:



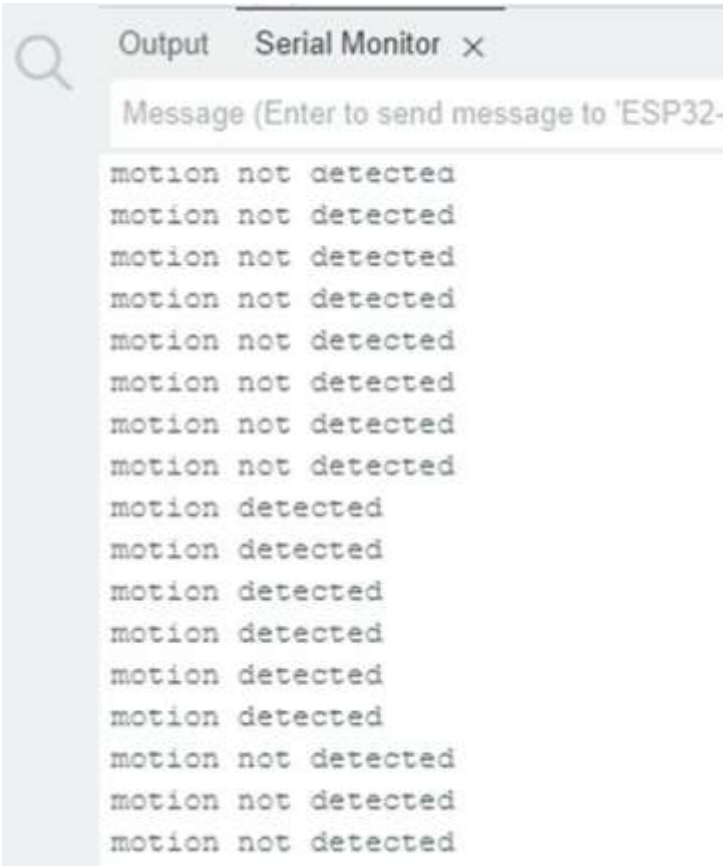
Connection diagram:



Code:

```
#define IR_PIN 13
 void setup() {
Serial.begin(9600);
pinMode(IR_PIN, INPUT);
pinMode(15, OUTPUT);  //Buzzer
}
void loop() {
if (digitalRead(IR_PIN) == LOW)
{
Serial.println("Motion detected");
digitalWrite(15,HIGH);
}
else{
Serial.println("Motion not detected");
digitalWrite(15,LOW);
}
delay(200);
}
```

Sample Output on serial monitor:



Assignment–1 (Application-based Scenario):

In a security surveillance system, monitoring activity frequency is essential. Modify the code to count the number of motion detections within a one-minute interval and display the count.

Code and output:

Assignment–2 (Simulation Task):

Simulate the existing IR sensor motion detection program in TinkerCAD by triggering motion events and verifying buzzer operation. Capture the simulation result.

About IR sensor: https://www.electronicsforu.com/technology-trends/learn-electronics/ir-led-infrared-sensor-basics

Applications:

-
-
-
-
-

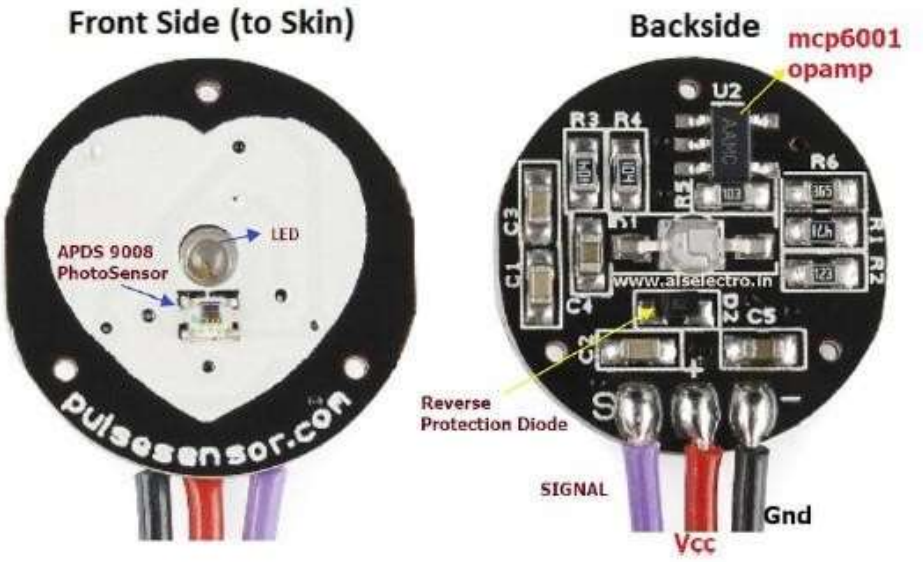
Work Sheet

Signature of Staff In-charge with date

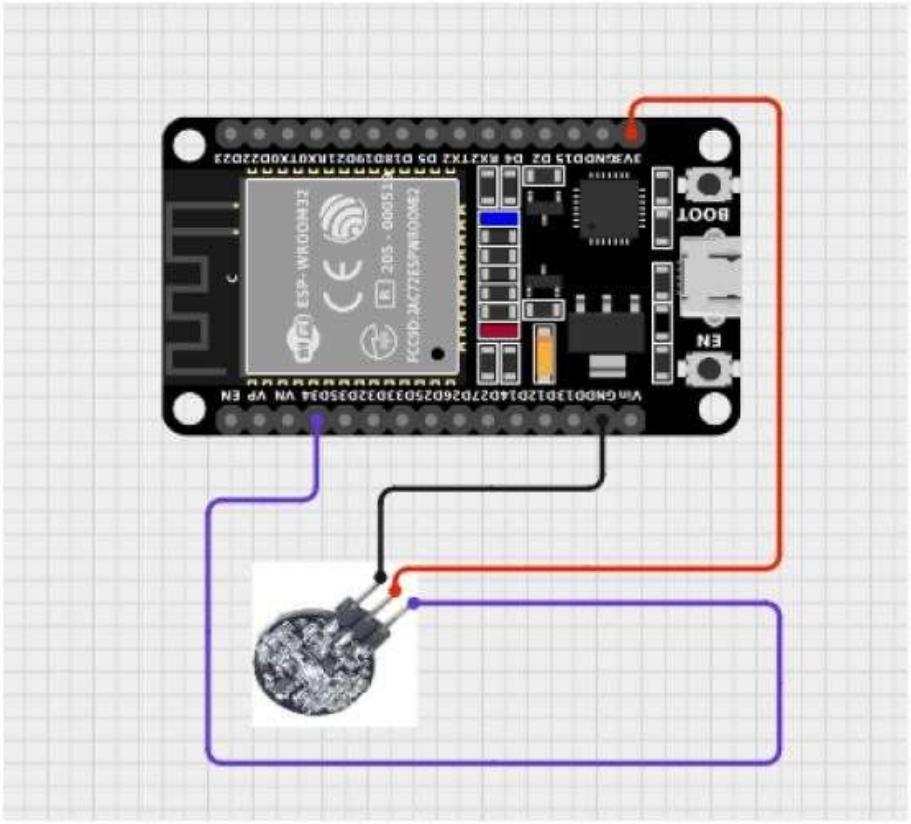
EXPERIMENT NO. 7 **Date:**

Aim: Design an interface circuit for Pulse Sensor with ESP32. Write a Program to display BPM value on serial monitor.

Pulse sensor:



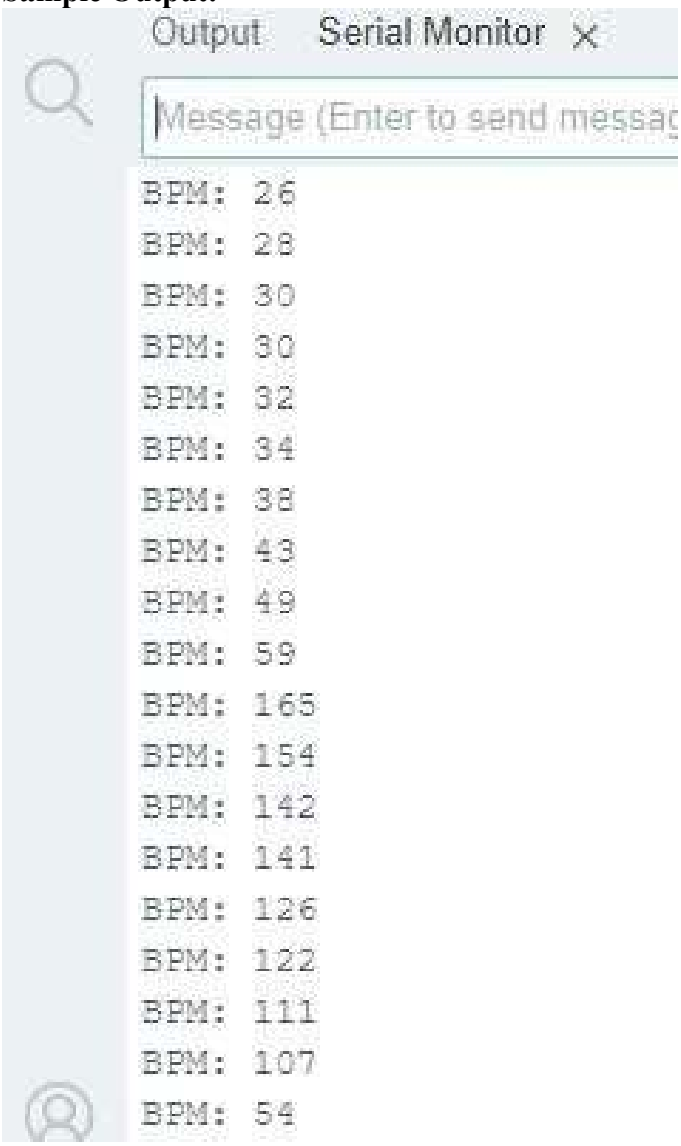
Connection diagram:



Code:

```
#include <PulseSensorPlayground.h>
#define PULSE_SENSOR_PIN 34 // Analog pin connected to pulse
//sensor
 #define THRESHOLD 550   // Adjust based on your sensor reading

PulseSensorPlayground pulseSensor; // Create PulseSensor

object void setup()
{
Serial.begin(115200);// Configure the pulse //sensor
pulseSensor.analogInput(PULSE_SENSOR_PIN);
pulseSensor.setThreshold(THRESHOLD);

if (pulseSensor.begin())
{
Serial.println("Pulse sensor initialized!");
}
else
{
Serial.println("Failed to initialize pulse sensor.");
}
}

void loop()
{
int myBPM = pulseSensor.getBeatsPerMinute(); // Get BPM
//value
 if (pulseSensor.sawStartOfBeat())
{
// If a new beat is detected
Serial.print("BPM: ");
Serial.println(myBPM);
}
delay(20); // Small delay to stabilize readings
}
```

Sample Output:



Assignment–*1 (Application-based Scenario):*

In wearable health monitoring systems, early detection of abnormal heart rates is critical. Modify the program to classify BPM values as Low, Normal, or High and display the health status.

Code and Output:

Assignment–2 (Simulation Task):

Simulate the existing BPM measurement logic using a virtual input source in TinkerCAD and observe BPM computation. Capture the simulation result.

BPM Sensor working: https://youtu.be/DNTfg_Uyvr0?si=D2U9Rh-BRl-1TAzj

Applications:

-
-
-
-
-

Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 8 a) Date:

Aim: Configure/Set your ESP32 development board as an access point.

Description: When you set your ESP32 board as an access point, you can be connected using any device with Wi-Fi capabilities without connecting to your router. When you set the ESP32 as an access point, you create its own Wi-Fi network, and nearby Wi-Fi devices (stations) can connect to it, like your smartphone or computer. So, you don't need to be connected to a router to control it.

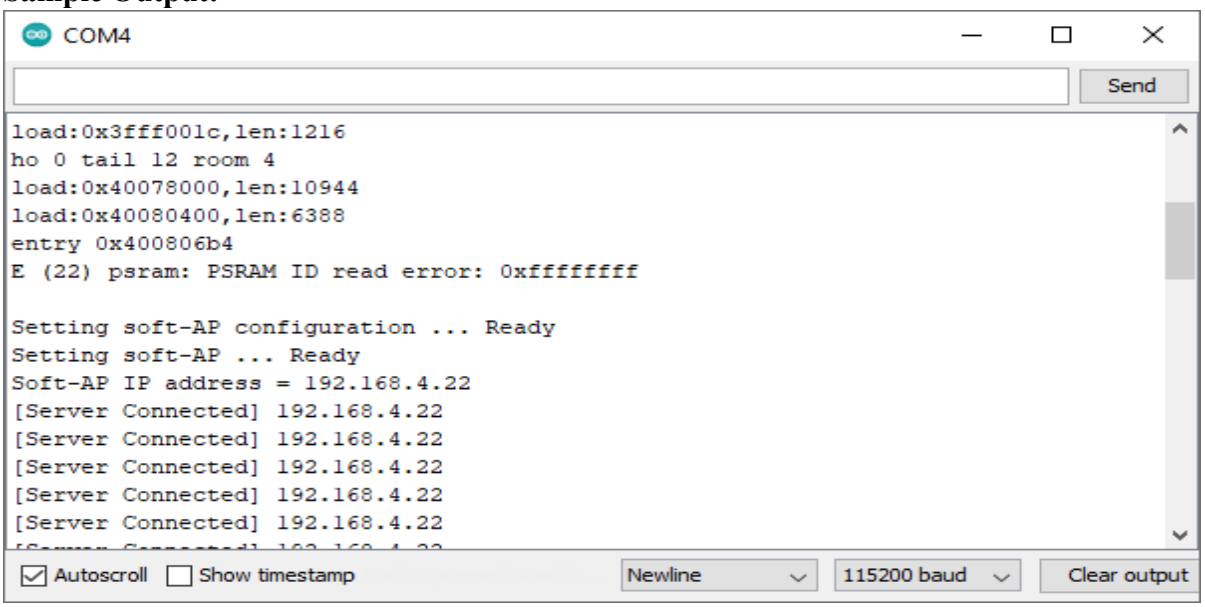
Procedure:

1. Plug the ESP32 development board to your PC
2. Open the Arduino IDE in computer and write the program. Save the new sketch in your working directory
   Note: **Make sure you have the right board and COM port selected in your Arduino IDE settings**.
3. Define a SSID name and a password to access the ESP32 development board
4. Compile the program and upload it to the ESP32 Development Board. If everything went as expected, you should see a "Done uploading" message. (You need to hold the ESP32 on-board Boot button while uploading).
5. After uploading the code, open the Serial Monitor at a baud rate of 115200.

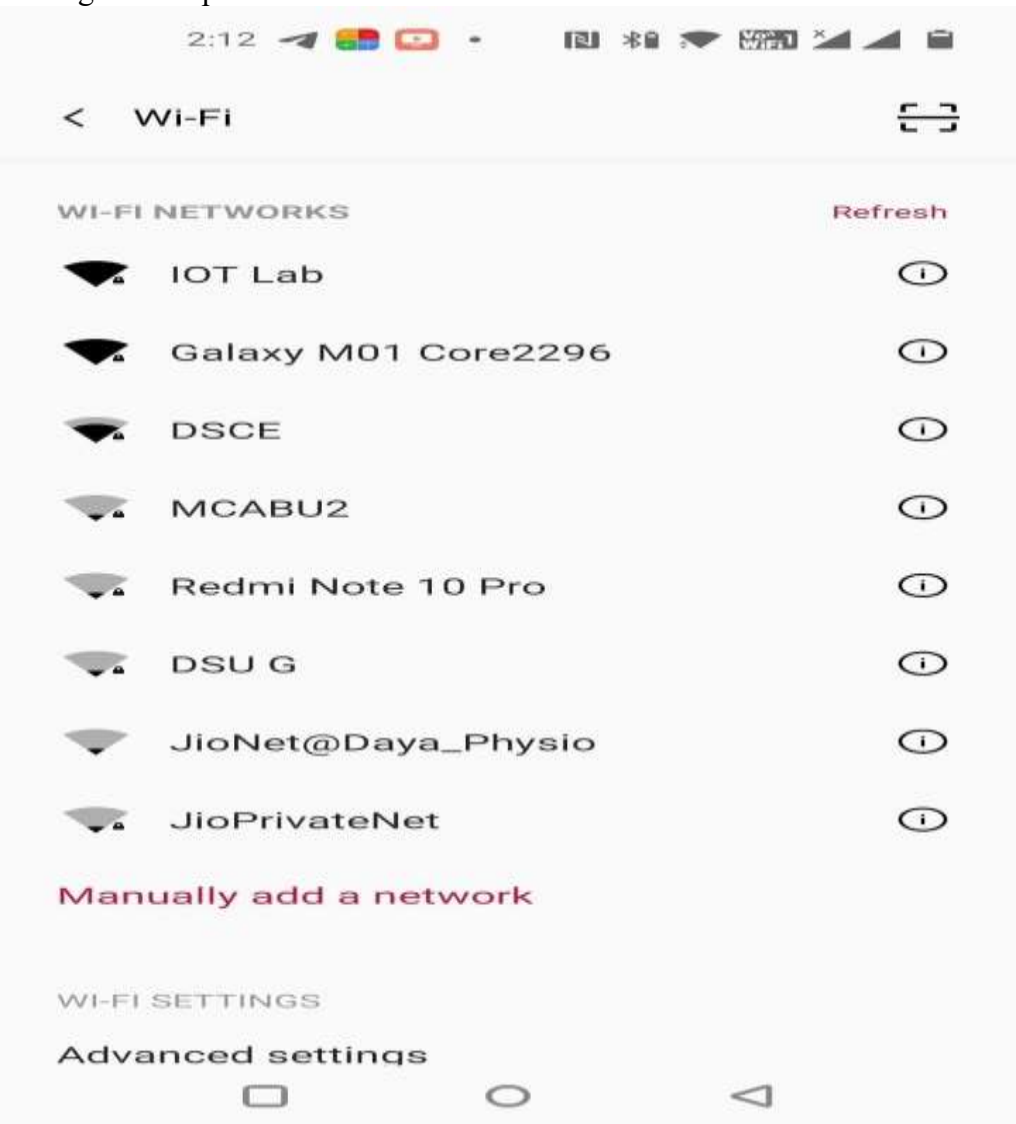
Code:

```
#include <WiFi.h>
const char *ssid = "SSID_Name_your_Choice";
const char *password = " Your_Choice(min 6 character) ";
IPAddress local_IP(192,168,4,22);
IPAddress          gateway(192,168,4,9);
IPAddress         subnet(255,255,255,0);
void setup()
{

 Serial.begin(115200);
 Serial.println();
 Serial.print("Setting soft-AP configuration ... ");
 Serial.println(WiFi.softAPConfig(local_IP, gateway, subnet) ? "Ready" : "Failed!");
 Serial.print("Setting soft-AP ... ");
 Serial.println(WiFi.softAP(ssid,password) ? "Ready" : "Failed!");
 //WiFi.softAP(ssid);
 //WiFi.softAP(ssid, password, channel, ssdi_hidden, max_connection)

 Serial.print("Soft-AP IP address = ");
 Serial.println(WiFi.softAPIP());
}
void loop() {
 Serial.print("[Server Connected] ");
 Serial.println(WiFi.softAPIP());
 delay(500);
}
```

Sample Output:



Through Smart phone:



Reference: https://www.electronicwings.com/esp32/esp32-wi-fi-basics-getting-started

Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 8 b) Date:

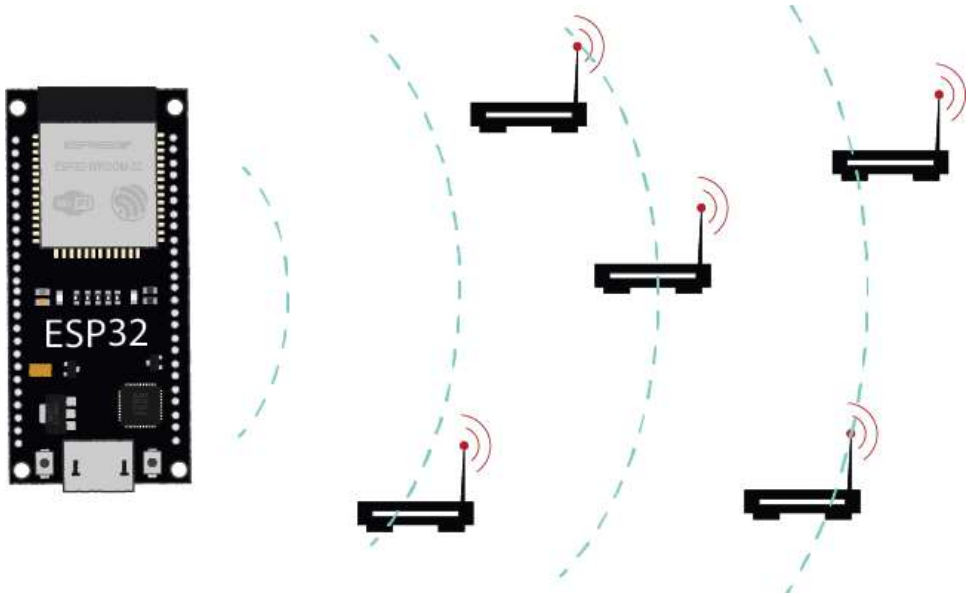
<u>Aim:</u> Scan Wi-Fi networks and get Wi-Fi strength using ESP32 development board.

ESP32 Wi-Fi Modes:
The ESP32 board can act as Wi-Fi Station, Access Point or both. To set the Wi-Fi mode, use WiFi.mode() and set the desired mode as argument:

WiFi.mode(WIFI_STA)	**station mode:** the ESP32 connects to an access point
WiFi.mode(WIFI_AP)	**access point mode:** stations can connect to the ESP32
WiFi.mode(WIFI_STA_AP)	**access point and a station** connected to another access point

Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	Micro USB cable	1



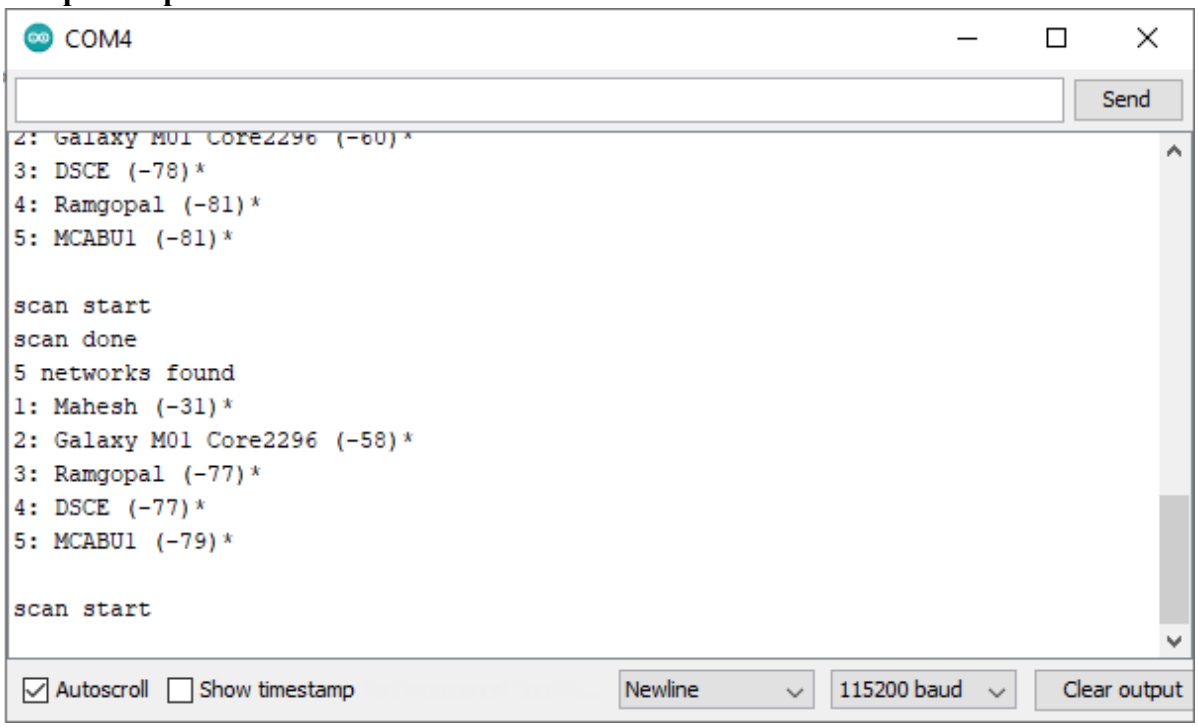
Procedure:

1. Plug the ESP32 development board to your PC
2. Make the connection as per the connection diagram
3. Open the Arduino IDE in computer and write the program. Save the new sketch in your working directory
   Note: **Make sure you have the right board and COM port selected in your Arduino IDE settings**.
4. Compile the program and upload it to the ESP32 Development Board. If everything went as expected, you should see a "Done uploading" message. (You need to hold the ESP32 on-board Boot button while uploading).
5. After uploading the code, open the Serial Monitor at a baud rate of 115200.

Code:

```
#include "WiFi.h"

void setup()
{
  Serial.begin(115200);

  // Set WiFi to station mode and disconnect from an AP if it was previously connected
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(100);

  Serial.println("Setup done");
}
void loop()
{
  Serial.println("scan start");

  // WiFi.scanNetworks will return the number of networks found int n =
  WiFi.scanNetworks();
  Serial.println("scan done");
  if (n == 0) {
    Serial.println("no networks found");
  } else {
    Serial.print(n);
    Serial.println(" networks found");
    for (int i = 0; i < n; ++i) {
      // Print SSID and RSSI for each network found
      Serial.print(i + 1);
      Serial.print(": ");
      Serial.print(WiFi.SSID(i));
      Serial.print(" (");
      Serial.print(WiFi.RSSI(i)); Serial.print(")");
      Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN)?" ":"*");
      delay(10);
    }
  }
  Serial.println("");

  // Wait a bit before scanning again
   delay(5000);
}
```

Sample Output:



Assignment (Application-based Scenario):

In intelligent network selection systems, devices must automatically choose the best available network. Modify the program to identify and display the Wi-Fi network with the strongest RSSI value.

Code and Output:

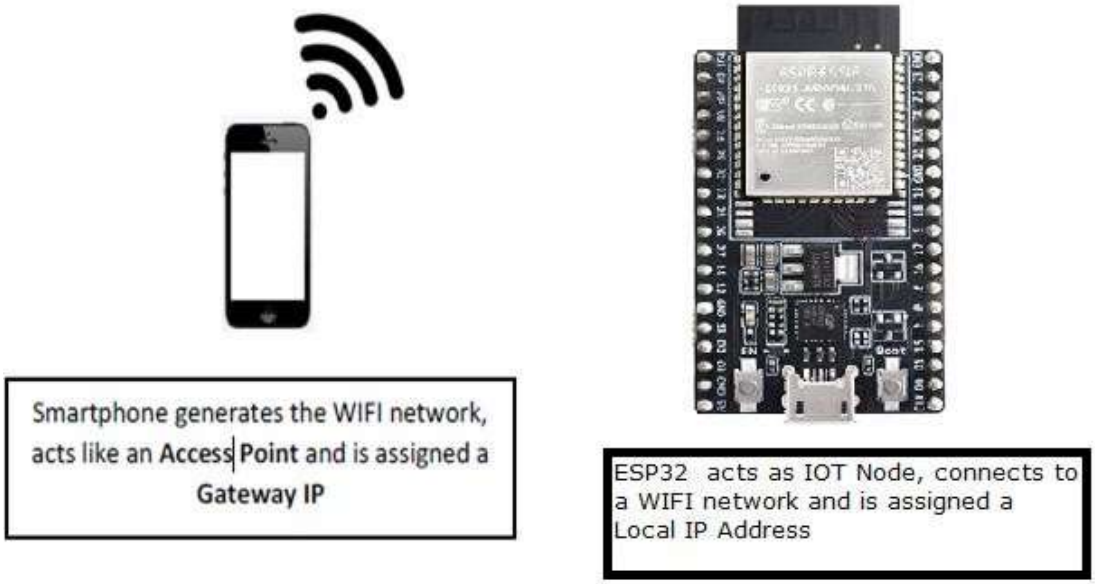
Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 8 c) Date:

<u>Aim:</u> Establish connection between IoT node and access point and display of local IP and gateway IP.

Description: The goal of this experiment is to understand the configuration of IoT Node to connect to an Access Point. Any device when connected to a network is assigned an IP address, which is a unique number for each device on a network. We shall also see how to identify the IP network of our IoT Node. The Access Point also has an IP address, known as the Gateway IP. These concepts are fundamentals to understand the architecture of an IoT Network.



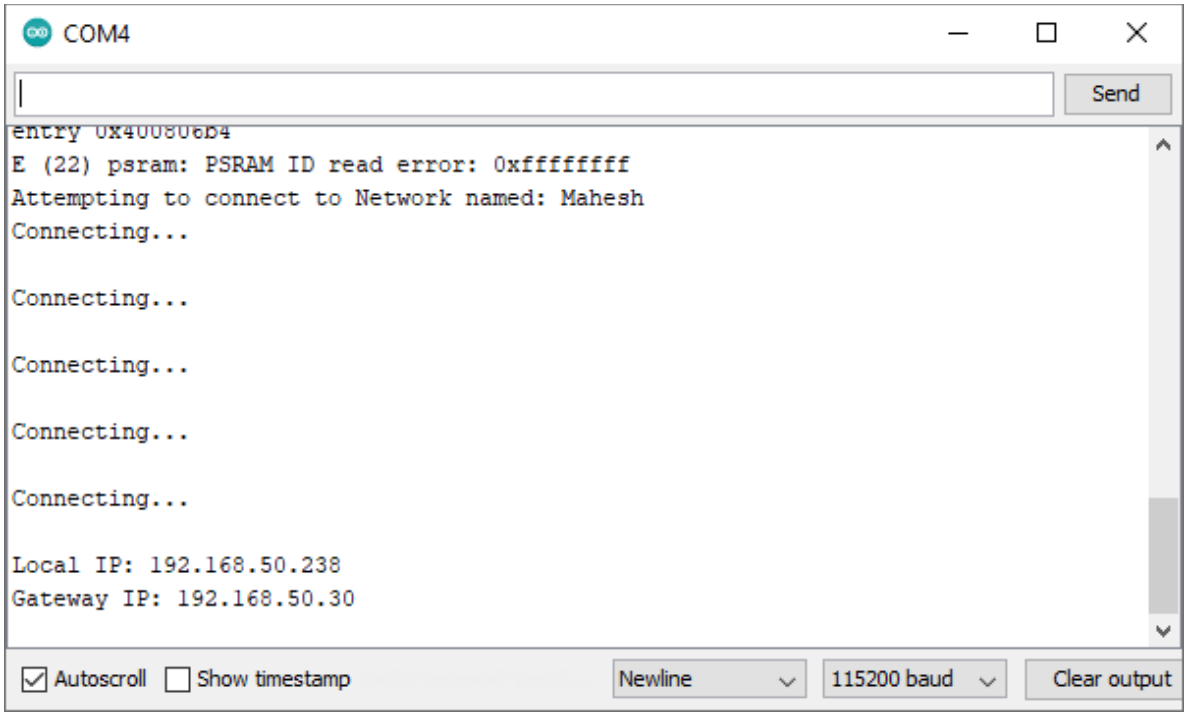
Procedure:

1. Plug the ESP32 development board to your PC
2. Open the Arduino IDE in computer and write the program. Save the new sketch in your working directory
   Note: **Make sure you have the right board and COM port selected in your Arduino IDE settings**.
3. Define a SSID name and a password to access the ESP32 development board
4. Compile the program and upload it to the ESP32 Development Board. If everything went as expected, you should see a "Done uploading" message. (You need to hold the ESP32 on-board Boot button while uploading).
5. After uploading the code, open the Serial Monitor at a baud rate of 115200.

Code:

```
#include <WiFi.h>

char ssid[]="REPLACE_WITH_YOUR_SSID";
char password[]="REPLACE_WITH_YOUR_PASSWORD";
IPAddress ip;
IPAddress gateway;

void setup()
{
 Serial.begin(115200); //Initialize Serial Port
 //attempt to connect to wifi
 Serial.print("Attempting to connect to Network named: ");
 // print the network name (SSID);
 Serial.println(ssid);
  //Connect to WiFI
 WiFi.begin(ssid, password);

 //Wait untill wifi is connected
 while ( WiFi.status() != WL_CONNECTED)
 {
  // print dots while we wait to connect
  Serial.print(".");
  delay(300);
 }

 //If you are connected print the IP Address
 Serial.println("\nYou're connected to the network");
  //wait untill you get an IP address
 while (WiFi.localIP() = INADDR_NONE) {
 // print dots while we wait for an ip addresss
 Serial.print(".");
 delay(300);
 }

 ip=WiFi.localIP();
 Serial.println(ip);
 gateway=WiFi.gatewayIP();
 Serial.println("GATEWAY IP:");
 Serial.println(gateway);
}

void loop()
{
 // put your main code here, to run repeatedly:
}
```

Sample Output:



Assignment–3 (Application-based Networking Scenario):

In next-generation IoT networks, IPv6 is increasingly adopted to overcome IPv4 address exhaustion. Modify the existing program to extract the assigned local IPv4 address and compute its IPv6-mapped address representation (IPv4-mapped IPv6 format) and display both addresses on the Serial Monitor.

Code and Output:

Work Sheet

Signature of Staff In-charge with date

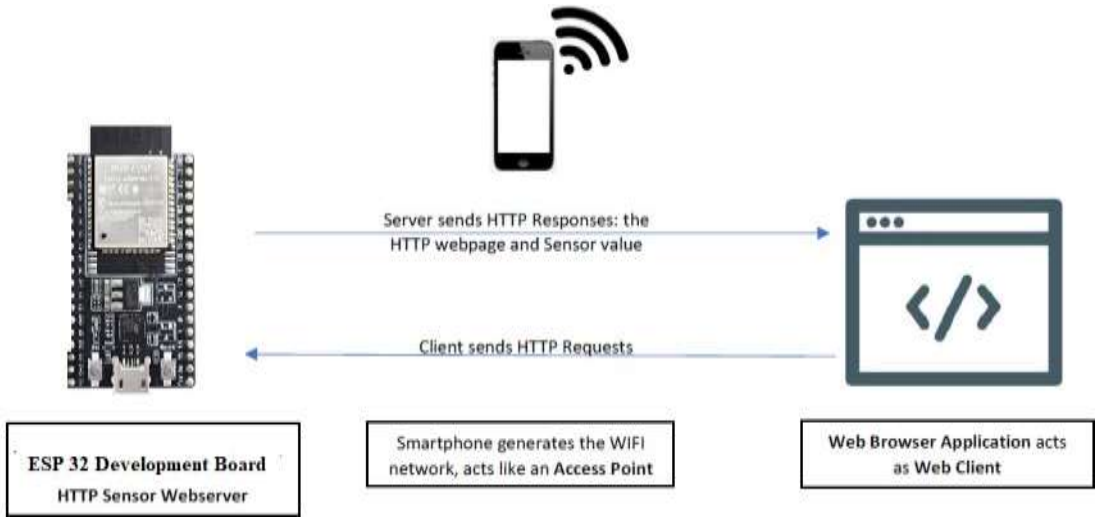
EXPERIMENT NO. 9 Date:

<u>Aim:</u> Design and implementation of HTTP based IoT web server to control the status of LED.

Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	DHT11 or DHT22 temperature and humidity sensor	1
3	Jumper wires	3
4	Micro USB cable	1

Description: An HTTP based webserver provides access to data to an HTTP client such as web browser. In IoT applications, the IoT Nodes are connected to sensors and actuators. The sensor data can be accessed using a web browser and also the actuators can be controlled from the web browser. This facilitates a wide variety of applications in IoT domain. The underlying protocol used for this communication between a Client and a Server is HTTP. The goal of this lab is to understand the configuration and working principle of an IoT web server. Using a browser, such as google chrome, we will send some HTTP based commands to the web server. Based on the type of command, the IoT Node web server will toggle the LEDs.

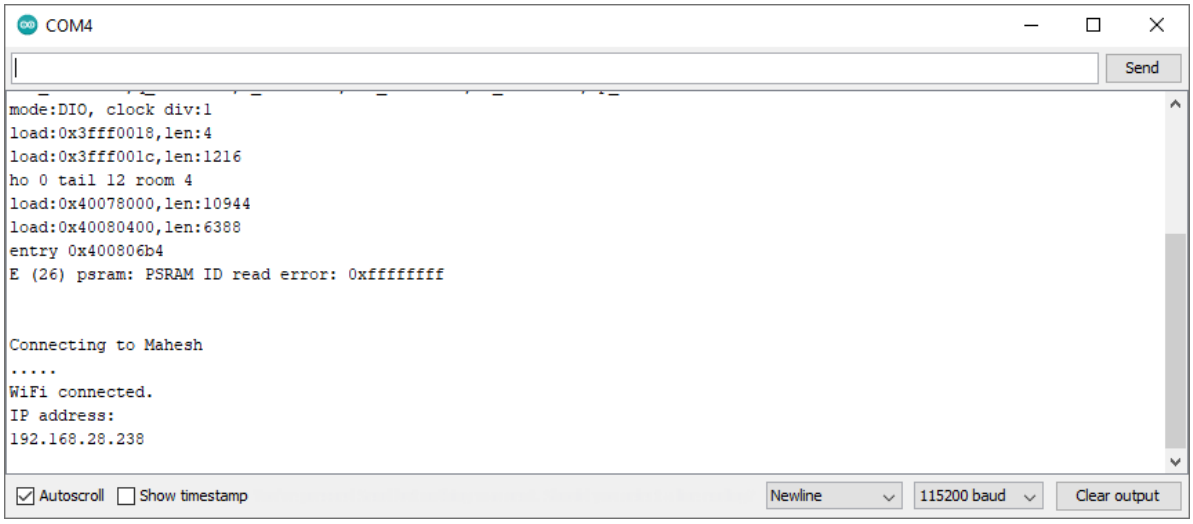


Code:

```
#include <WiFi.h>

const char* ssid     = "Mahesh";
const char* password = "012345678";

WiFiServer server(80);

void setup()
{
   Serial.begin(115200);
   pinMode(15, OUTPUT);    // set the External LED pin  mode

   delay(10);

   // We start by connecting to a WiFi network

   Serial.println();

   Serial.println();
   Serial.print("Connecting to ");
   Serial.println(ssid);

   WiFi.begin(ssid, password);

   while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
   }

   Serial.println("");
   Serial.println("WiFi connected.");
    Serial.println("IP address: ");
   Serial.println(WiFi.localIP());

   server.begin();
}

int value = 0;

void loop(){
 WiFiClient client = server.available();   // listen for incoming clients

  if (client) {                             // if you get a client,
   Serial.println("New Client.");           // print a message out the serial port
   String currentLine = "";                // make a String to hold incoming data from the
   //client
   while (client.connected()) {            // loop while the client's connected
```

```
    if (client.available()) {            // if there's bytes to read from the client,
      char c = client.read();             // read a byte, then
      Serial.write(c);                    // print it out the serial monitor
      if (c == '\n') {                    // if the byte is a newline character

        // if the current line is blank, you got two newline characters in a row.
        // that's the end of the client HTTP request, so send a response:
        if (currentLine.length() == 0) {
          // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
          // and a content-type so the client knows what's coming, then a blank line:
          client.println("HTTP/1.1 200 OK");
          client.println("Content type:text/html"); client.println();

          // the content of the HTTP response follows the header:
          client.print("Click <a href=\"/H\">here</a> to turn the LED on pin 2 on.<br>");
          client.print("Click <a href=\"/L\">here</a> to turn the LED on pin 2 off.<br>");

          // The HTTP response ends with another blank line:
          client.println();
          // break out of the while loop:
          break;
        }
        else
        {
        // if you got a newline, then clear currentLine:
          currentLine = "";
        }
      } else if (c != '\r') {  // if you got anything else but a carriage return
        character, currentLine += c;          // add it to the end of the currentLine
      }

      // Check to see if the client request was "GET /H" or "GET /L":
       if (currentLine.endsWith("GET /H")) {
        digitalWrite(15, HIGH);             // GET /H turns the LED on
      }
      if (currentLine.endsWith("GET /L")) {
        digitalWrite(15, LOW);              // GET /L turns the LED off
      }
    }
  }
  // close the connection:
  client.stop();
  Serial.println("Client Disconnected.");
 }
}
```

Sample output On serial monitor: (P.T.O)



On Web browser/Web Client:

Assignment (Simulation Task):

Simulate the traffic light control system using three LEDs in TinkerCAD by running the existing web server logic (or equivalent control logic) and verify correct sequencing and indication of traffic states. Capture the simulation result.

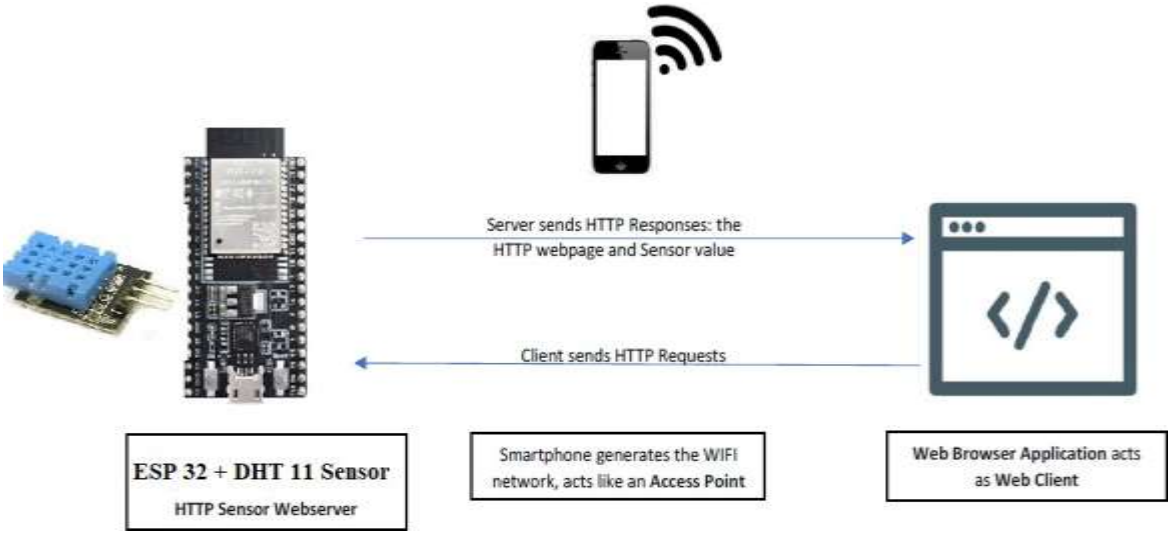
Work Sheet

Signature of Staff In-charge with date

EXPERIMENT NO. 10 **Date:**

Aim: Design and implementation of HTTP based IoT web server to display sensor value.

Description: The goal of this experiment is to integrate the HTTP webserver with Sensor. As we know, most IoT Nodes are equipped with sensors and the IoT systems should provide these sensor data to users. In this experiment, we used DHT11 Temperature and Humidity sensor connected to an IoT Node/ ESP32 development board. The IoT Node is configured as a Webserver and the sensor data can be accessed via a web browser.



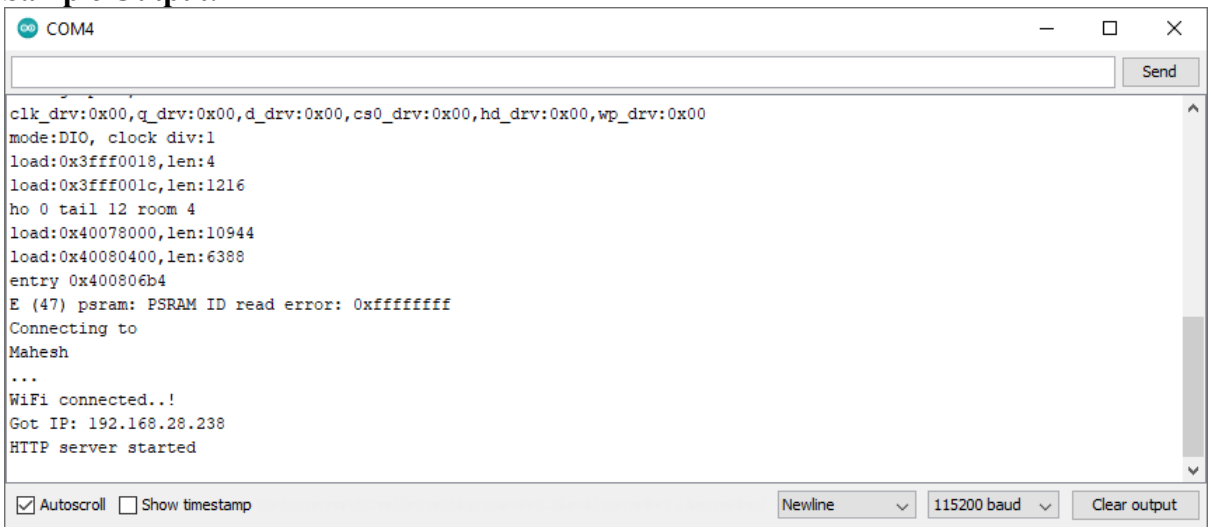
Code:

```
#include <WiFi.h>
 #include<WebServer.h>
 #include "DHT.h"

// Uncomment one of the lines below for whatever DHT
sensor type you're using!
#define DHTTYPE DHT11   // DHT 11
//#define DHTTYPE DHT21   // DHT 21 (AM2301)
//#define DHTTYPE DHT22   // DHT 22  (AM2302),
//AM2321

/*Put your SSID & Password*/
const char* ssid = " REPLACE_WITH_YOUR_SSID ";
// Enter SSID here
const char* password = “REPLACE_WITH_YOUR_PASSWORD";

//Enter Password here

WebServer server(80);

// DHT Sensor

uint8_t DHTPin = 4;

// Initialize DHT sensor.
DHT dht(DHTPin, DHTTYPE);

float Temperature;
float Humidity;

void setup() { Serial.begin(115200);
delay(100);

  pinMode(DHTPin, INPUT);

  dht.begin();

  Serial.println("Connecting to "); Serial.println(ssid);

  //connect to your local wi-fi network
  WiFi.begin(ssid, password);

  //check wi-fi is connected to wi-fi network
  while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
```

```
  Serial.print(".");
  }

  Serial.println("");
  Serial.println("WiFiconnected..!");

  Serial.print("Got IP: ");
  Serial.println(WiFi.localIP());

  server.on("/", handle_OnConnect);
  server.onNotFound(handle_NotFound);

  server.begin();
  Serial.println("HTTP server started");

}
void loop() { server.handleClient();

}

void handle_OnConnect() {

 Temperature = dht.readTemperature(); // Gets the
  values of the temperature Humidity =
  dht.readHumidity(); // Gets the values of the humidity
  server.send(200, "text/html",
  SendHTML(Temperature,Humidity));
}

void handle_NotFound(){
  server.send(404, "text/plain", "Not found");
}

String SendHTML(float Temperaturestat,float
  Humiditystat){ String ptr = "<!DOCTYPE html>
  <html>\n";
  ptr +="<head><meta name=\"viewport\"
content=\"width=device-width, initial-scale=1.0, user-
scalable=no\">\n";
  ptr +="<title>ESP32 Weather Report</title>\n";
  ptr +="<style>html { font-family: Helvetica; display:
inline-block; margin: 0px auto; text-align: center;}\n";
  ptr +="body{margin-top: 50px;} h1 {color:
  #444444;margin: 50px auto 30px;}\n"; ptr +="p {font-
  size: 24px;color: #444444;margin-bottom: 10px;}\n";
  ptr     +="</style>\n";     ptr     +="</head>\n";     ptr
  +="<body>\n";
  ptr +="<div id=\"webpage\">\n";
```

```
  ptr +="<h1>ESP32 Weather Report</h1>\n";
  ptr +="<p>Temperature: ";
  ptr+=(int)Temperaturesat;
  ptr +="\xC2\xB0 C</p>";
  ptr +="<p>Humidity: ";
  ptr+=(int)Humiditystat;
  ptr +="%</p>";

  ptr+="</div>\n";
  ptr+="</body>\n";
  ptr+="</html>\n";
  return ptr;
}
```

Sample Output:



On Web browser/Web Client:

Work Sheet

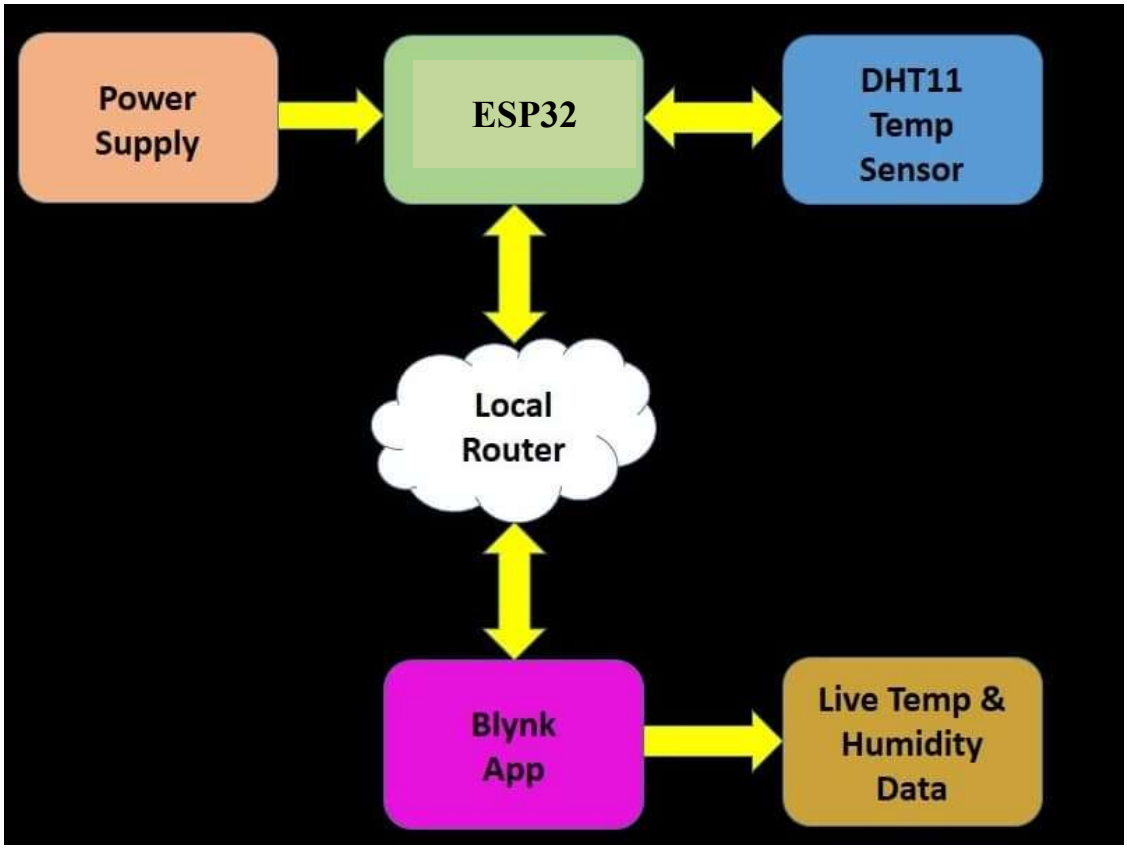
Signature of Staff In-charge with date

EXPERIMENT NO. 11 **Date:**

<u>Aim:</u> Design an interface for DHT11 (Temperature and Humidity) Sensor with ESP32. Write Program to display temperature and humidity on Blynk App. (Using Blynk Server).

Apparatus:

Sl No	Name of the Equipment	Quantity
1	ESP32 Development Board	1
2	DHT11 Sensor	1
3	Jumper wires	2
4	Micro USB cable	1

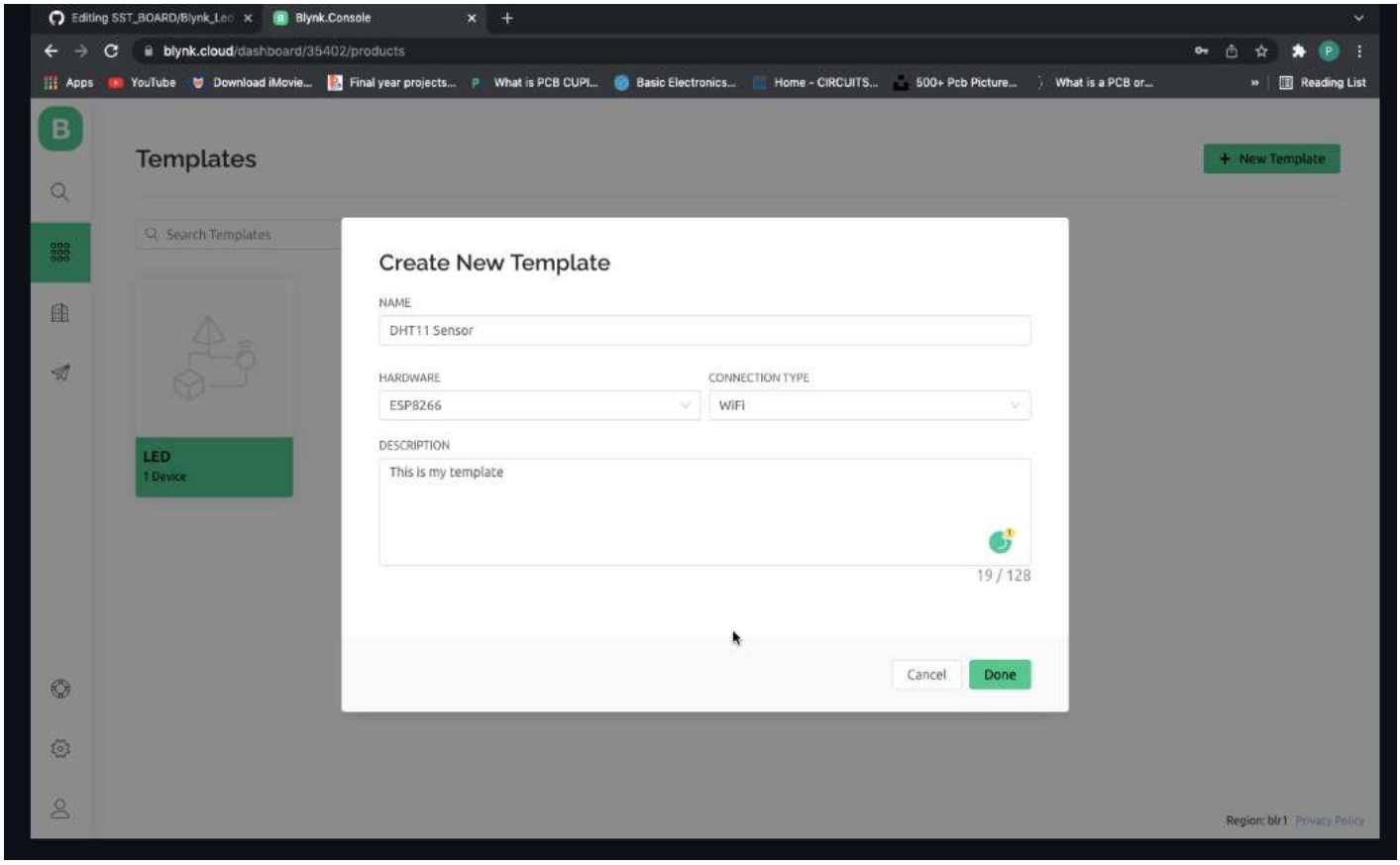


Create Account on Blynk Website:

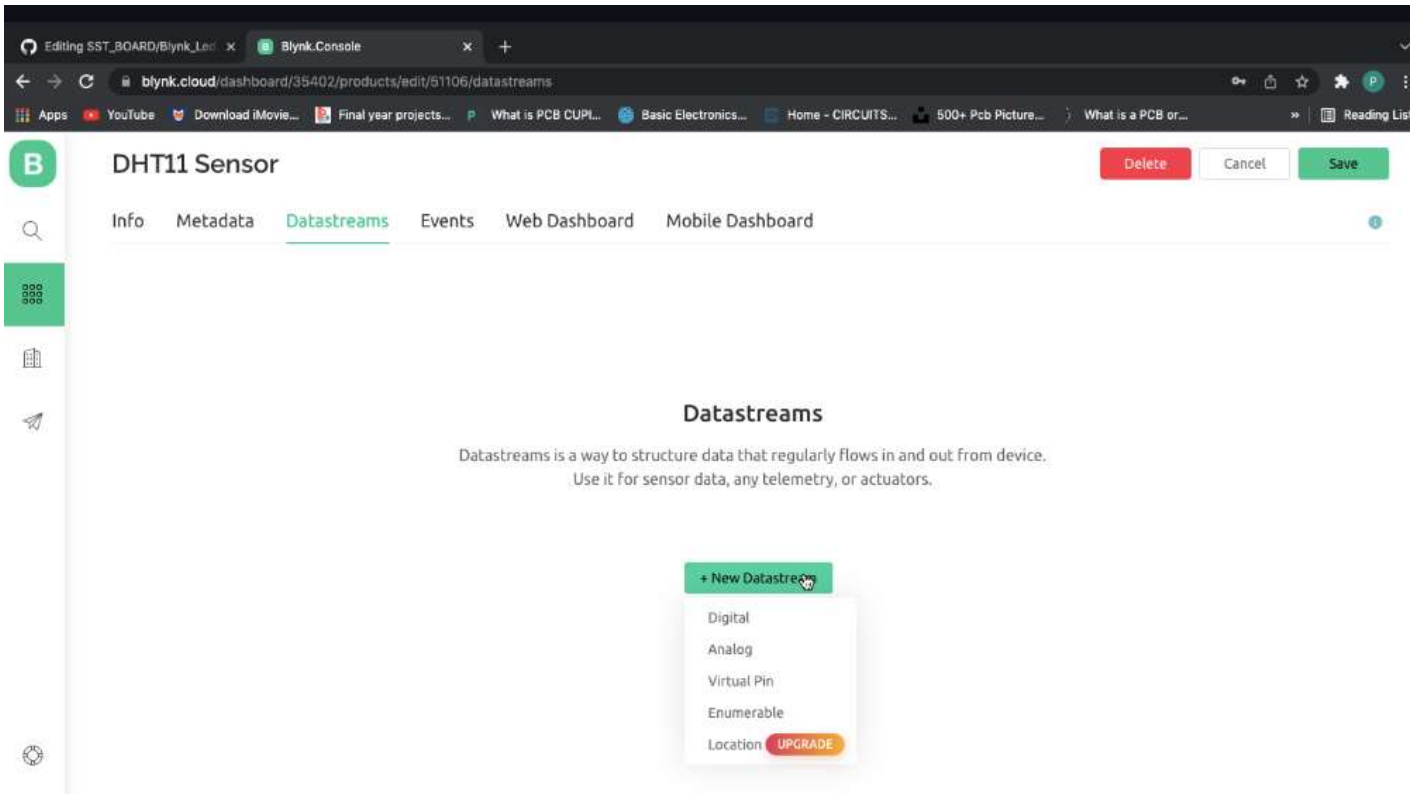
https://www.blynk.io/

Setup Guide: SST-IoT-BOARD/Blynk_DHT.md at c8f22eee5a7cc58aa1d4324318d7cb014c296cbd · izzarzn/SST-IoT-BOARD · GitHub

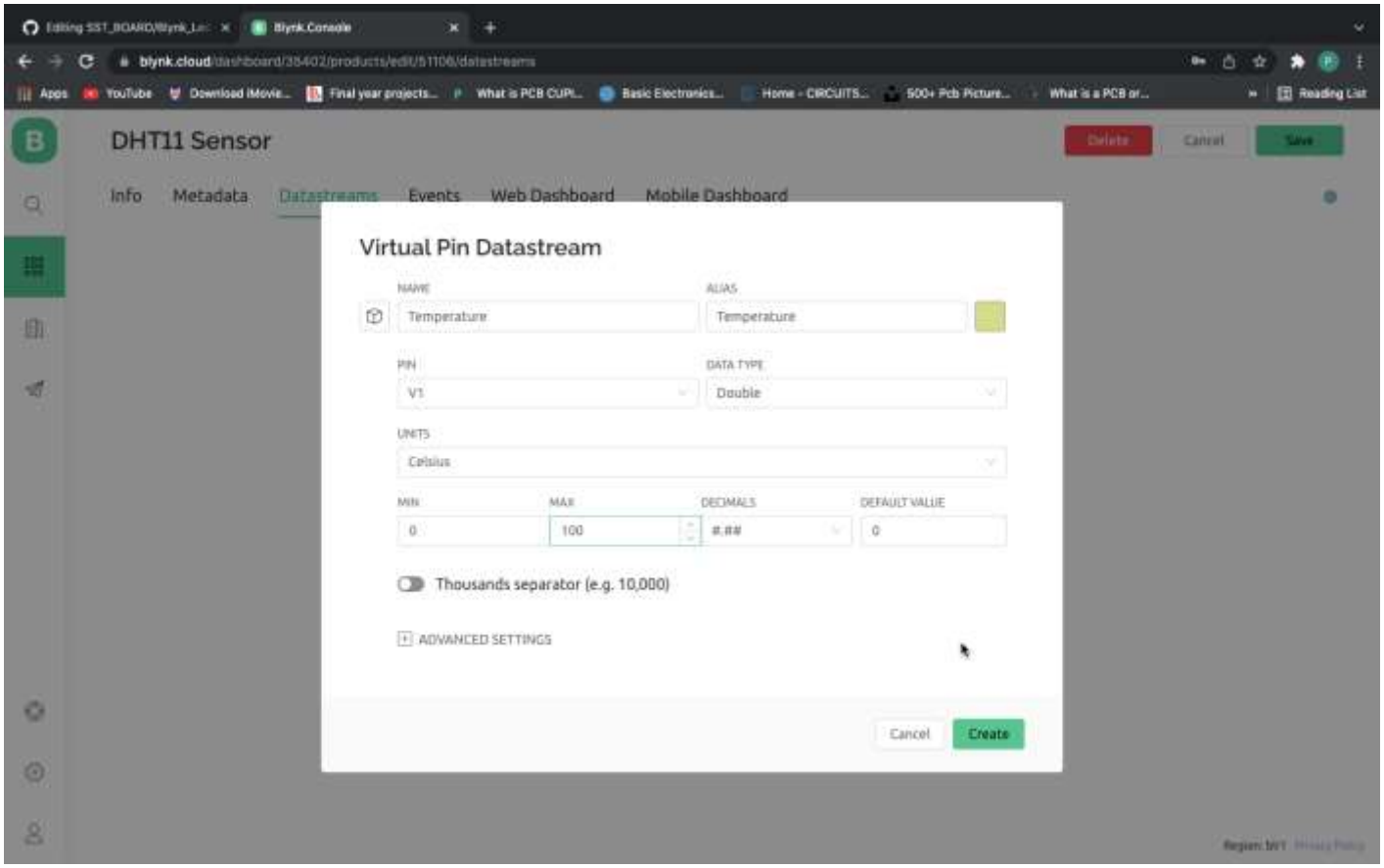
1. **Login to your Blynk Account. Click on New Template , fill in the details and click on Done.**



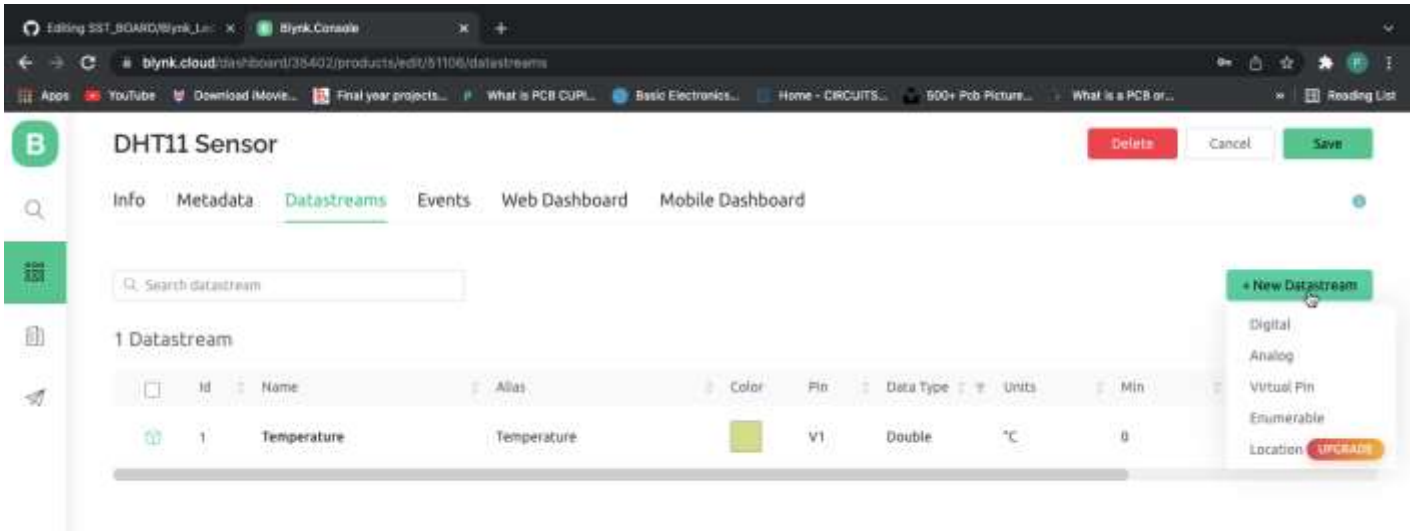
2. **Click On New Datatream in Datastreams and select Virtual Pin**



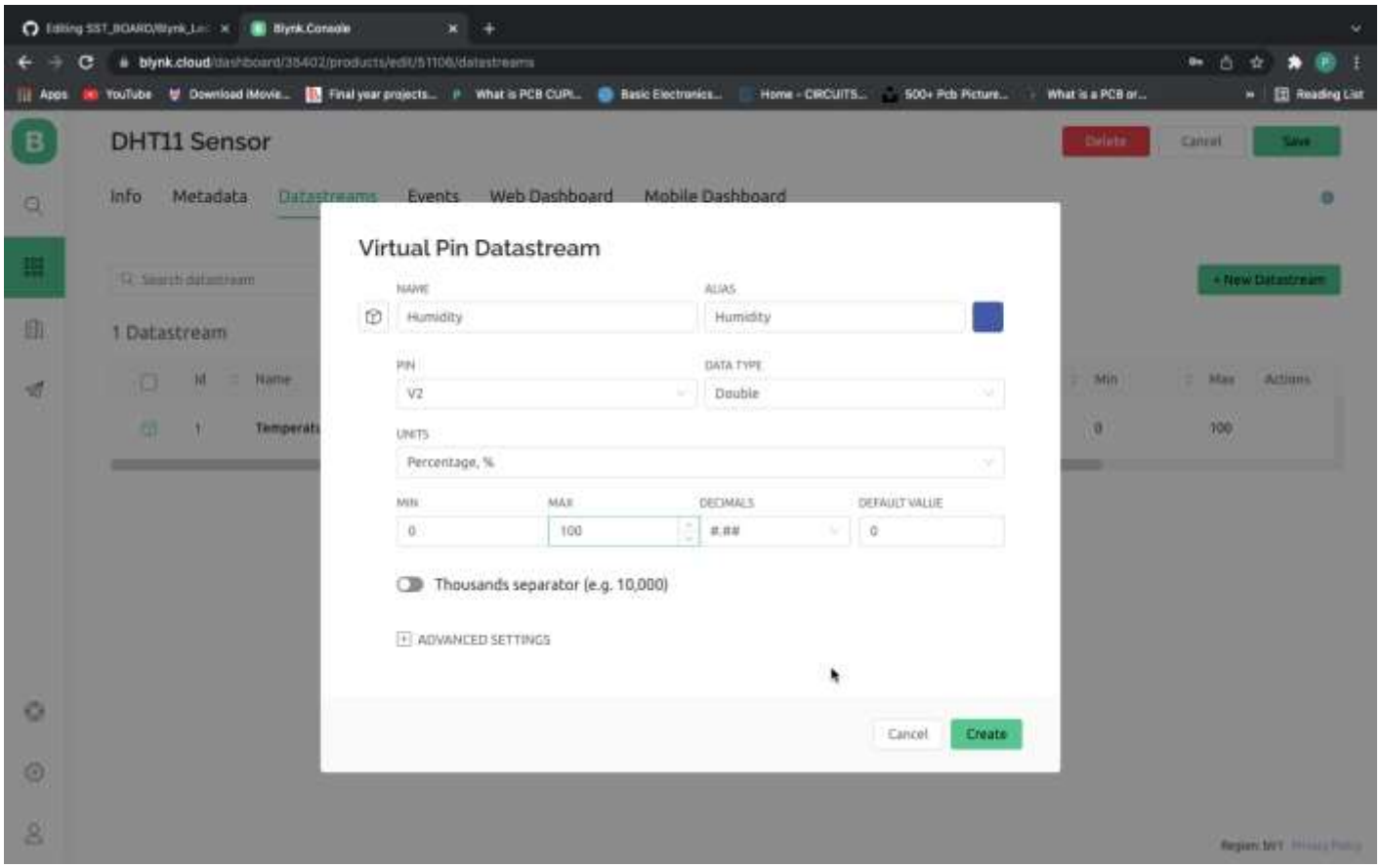
3. Fill in the Virtual Pin Details for Temperature and click on Create.



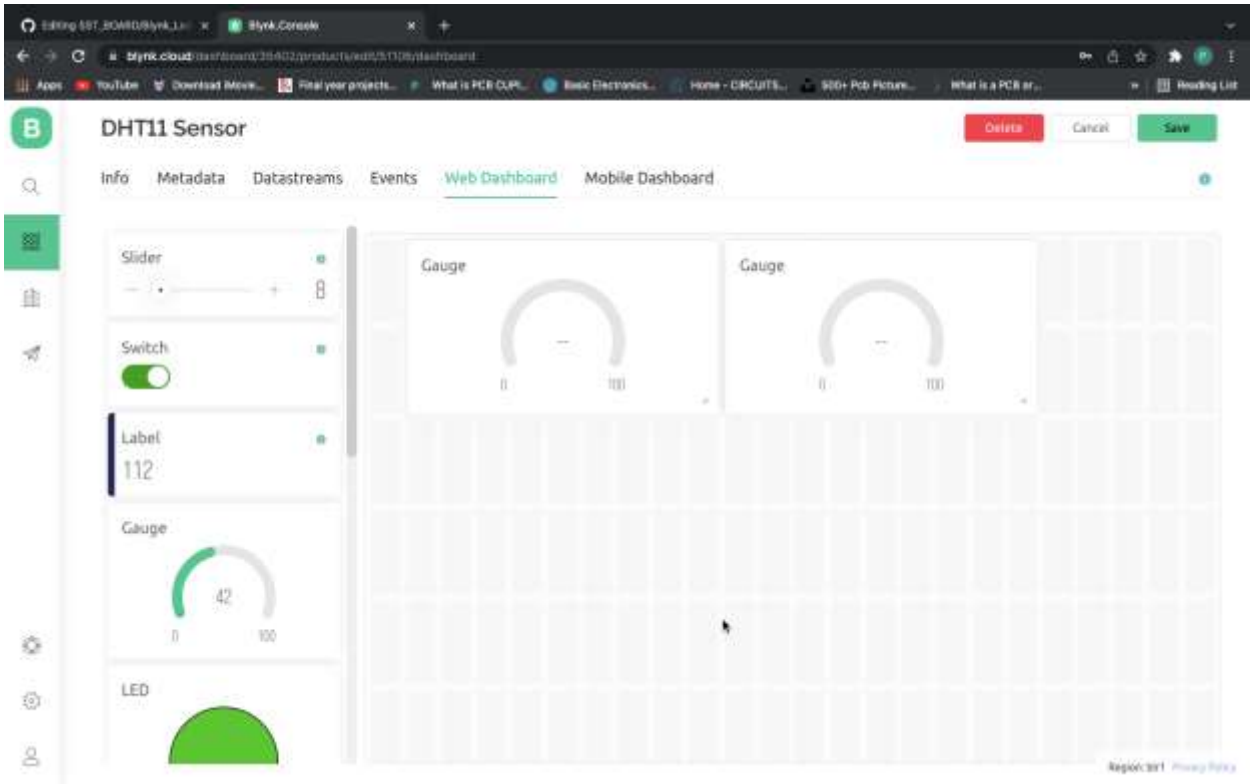
4. Click On New Datastream and select Virtual Pin



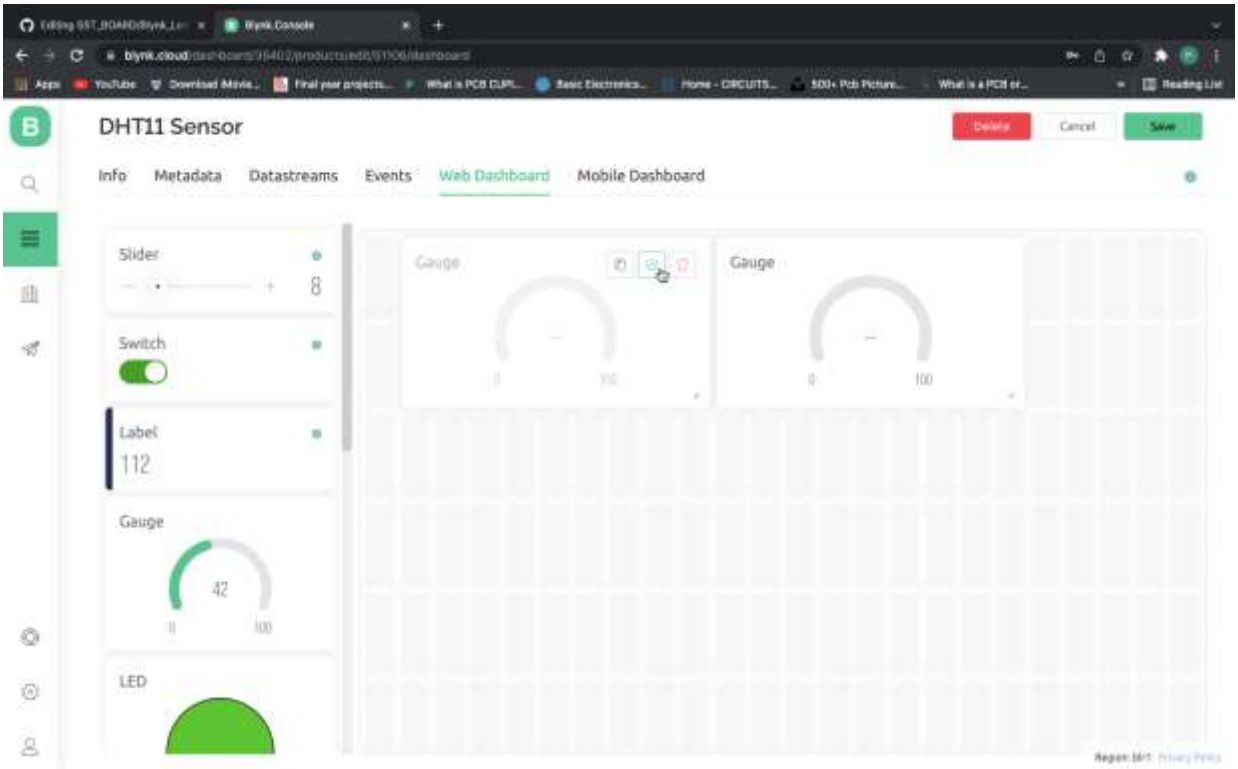
5. Fill in the Virtual Pin Details for Humidity and click on Create



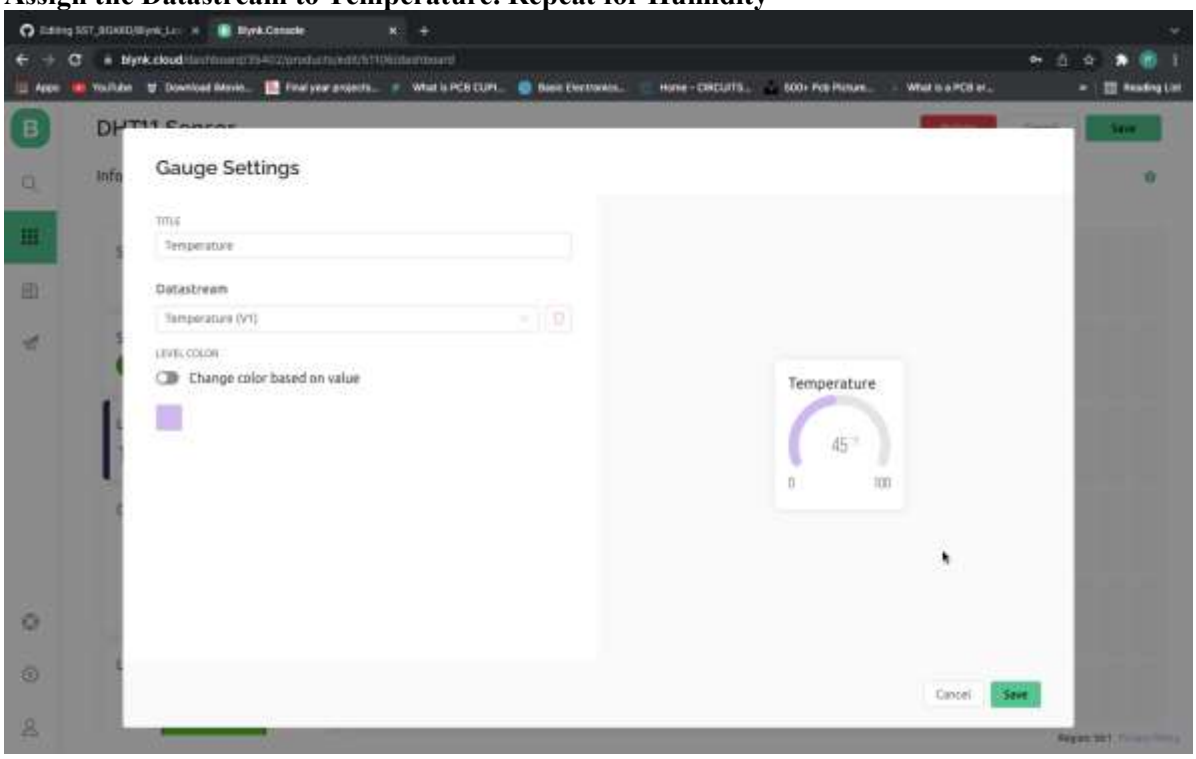
6. **Click On Web Dashboard and Drag two Guage Labels to right**



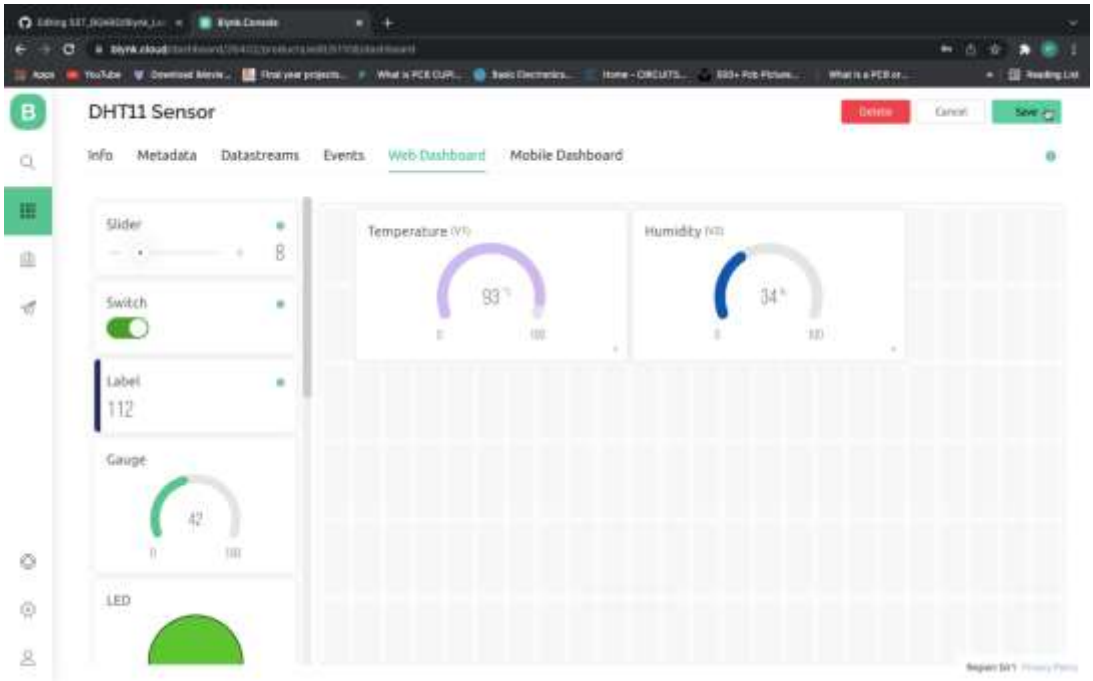
7. **Click On settings On the Label**



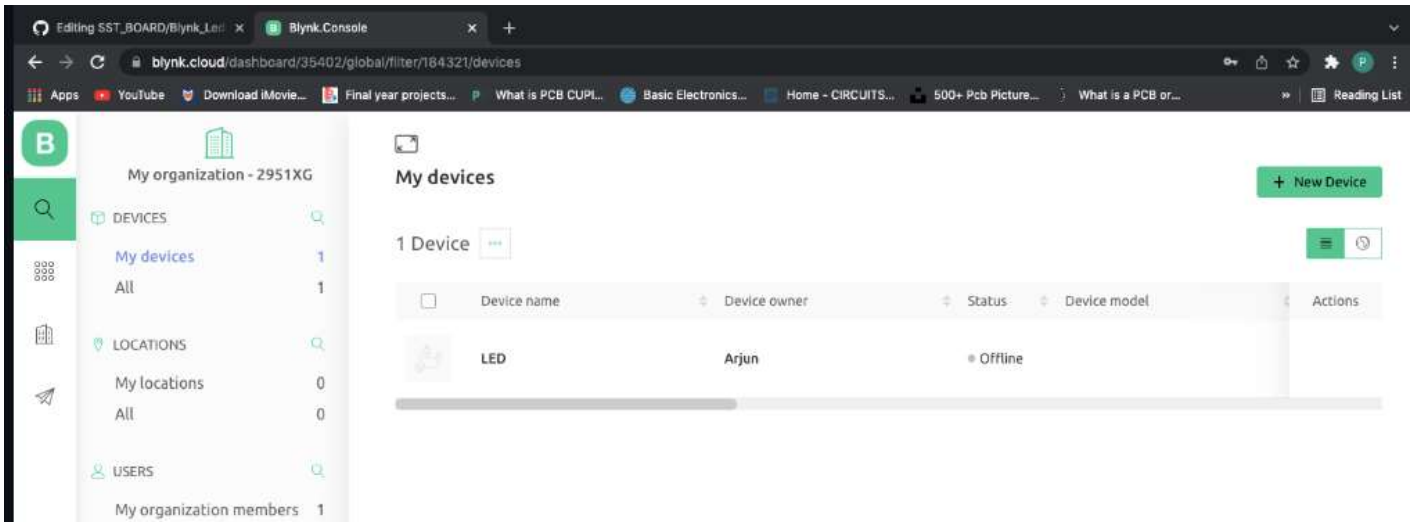
8. Assign the Datastream to Temperature. Repeat for Humidity



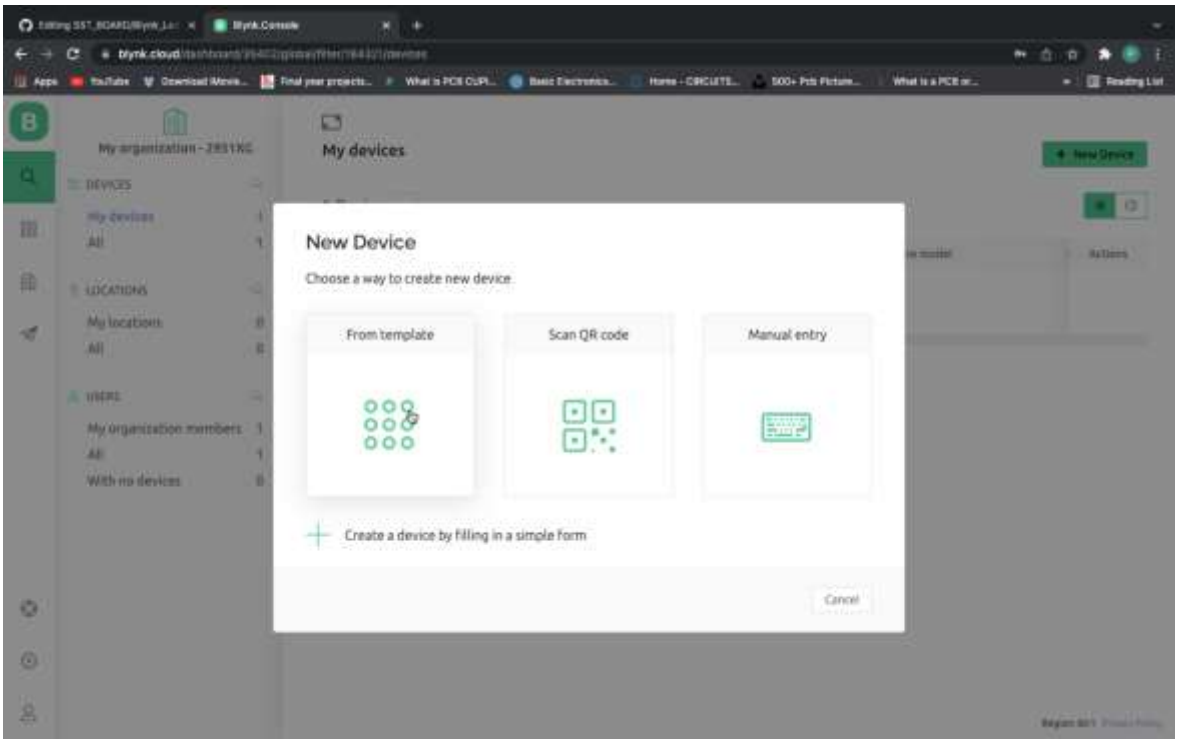
9. Click on Save



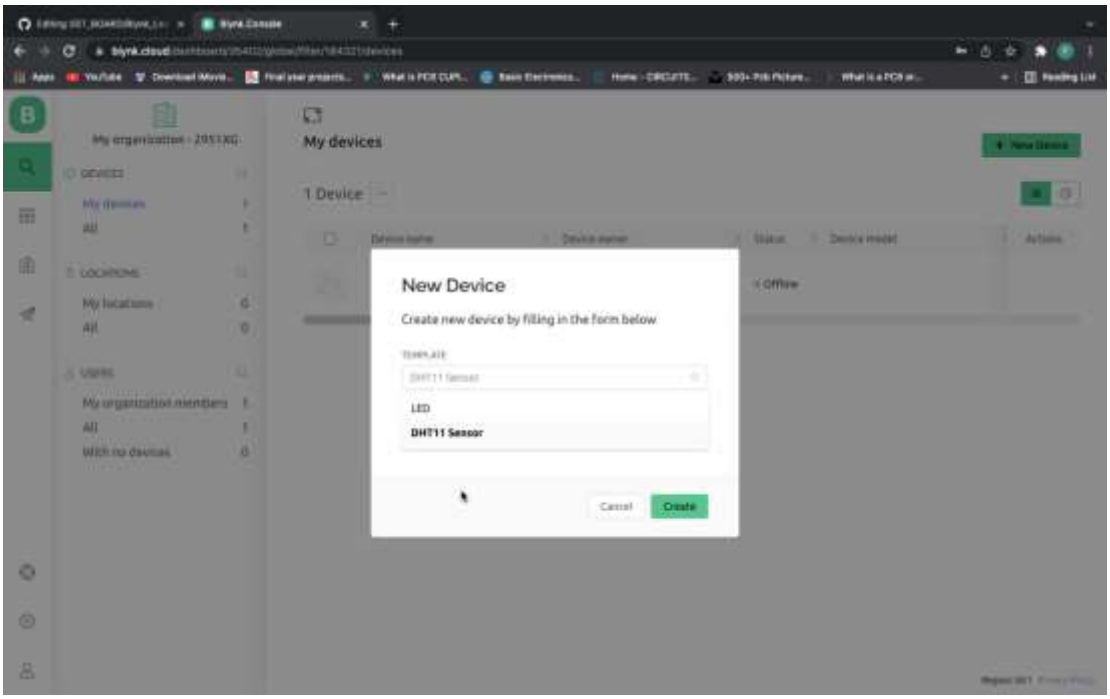
10. Go to Search and Click On New Device



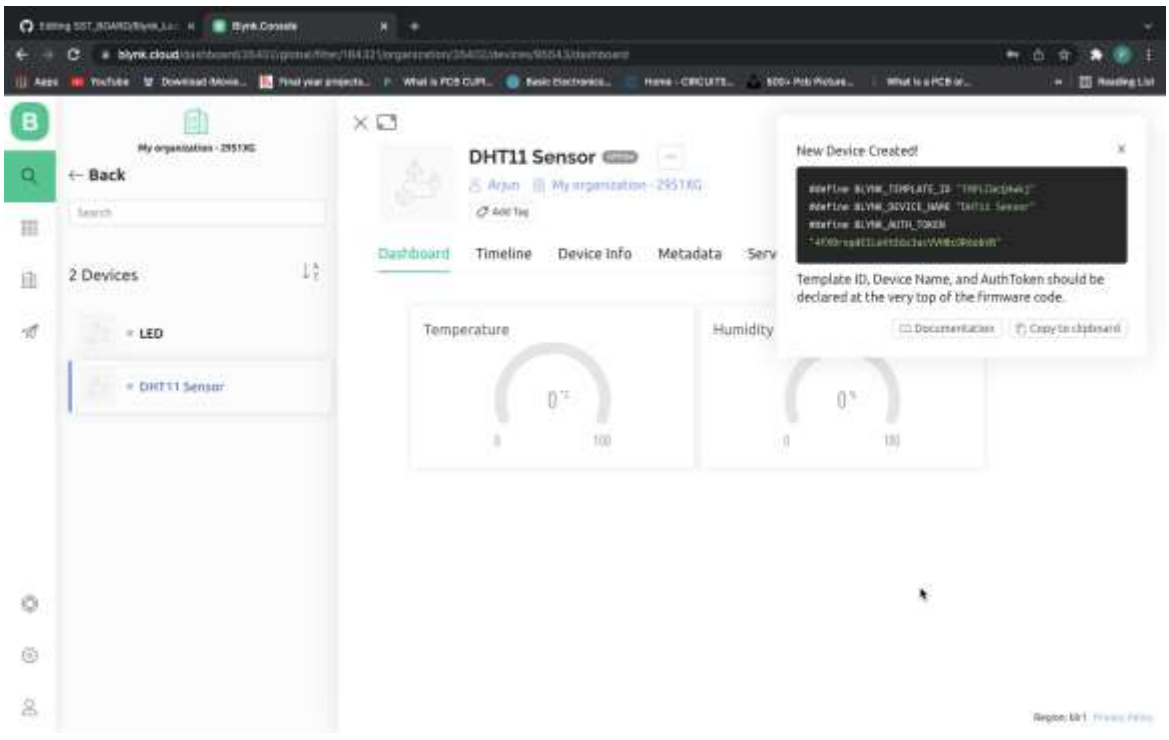
11. Click On From Template



12. Select the Template for which you want to add Device and Click on Create

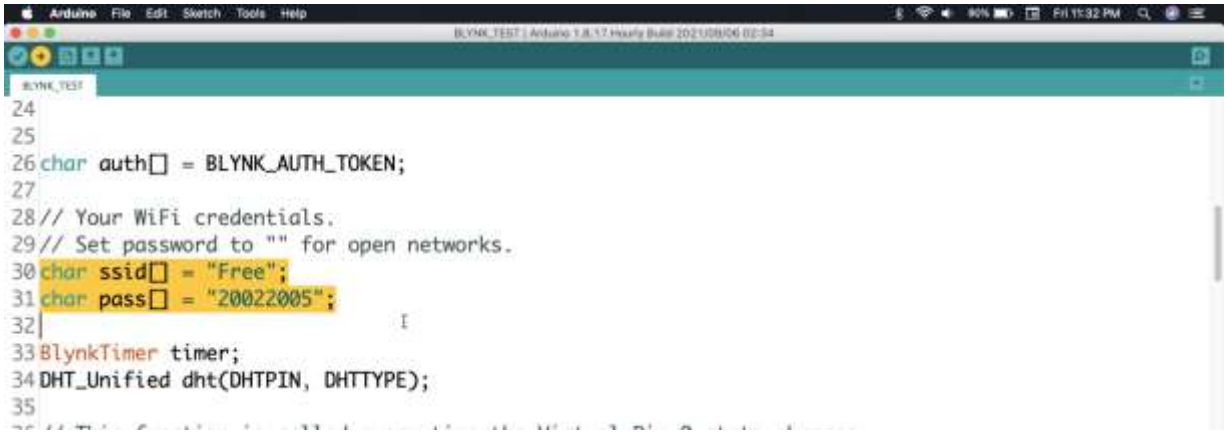


13. Copy the Code Displayed on Your right Side



14. Replace the Template ID Device Name and Auth Token in Your Program

15. Input your SSID and Password and Upload the code



Procedure :

1. Connect ESP32 to computer via USB .
2. Connect DHT11 Sensor to ESP32 as in Experiment -2.
3. Download the DHT Sensor Library, Adafruit Sensor, Blynk library. To Install these libraries in your Arduino IDE: go to Sketch >> Include Library >> Add. ZIP library and select the library you've just downloaded.
4. Go through the Setup Guide for creating the Blynk Server before uploading the Code.
5. Write the Program using Arduino-IDE and Verify (compile), and upload the code to the Board. Observe the Temperature and Humidity on your Web Dashboard.

Code:

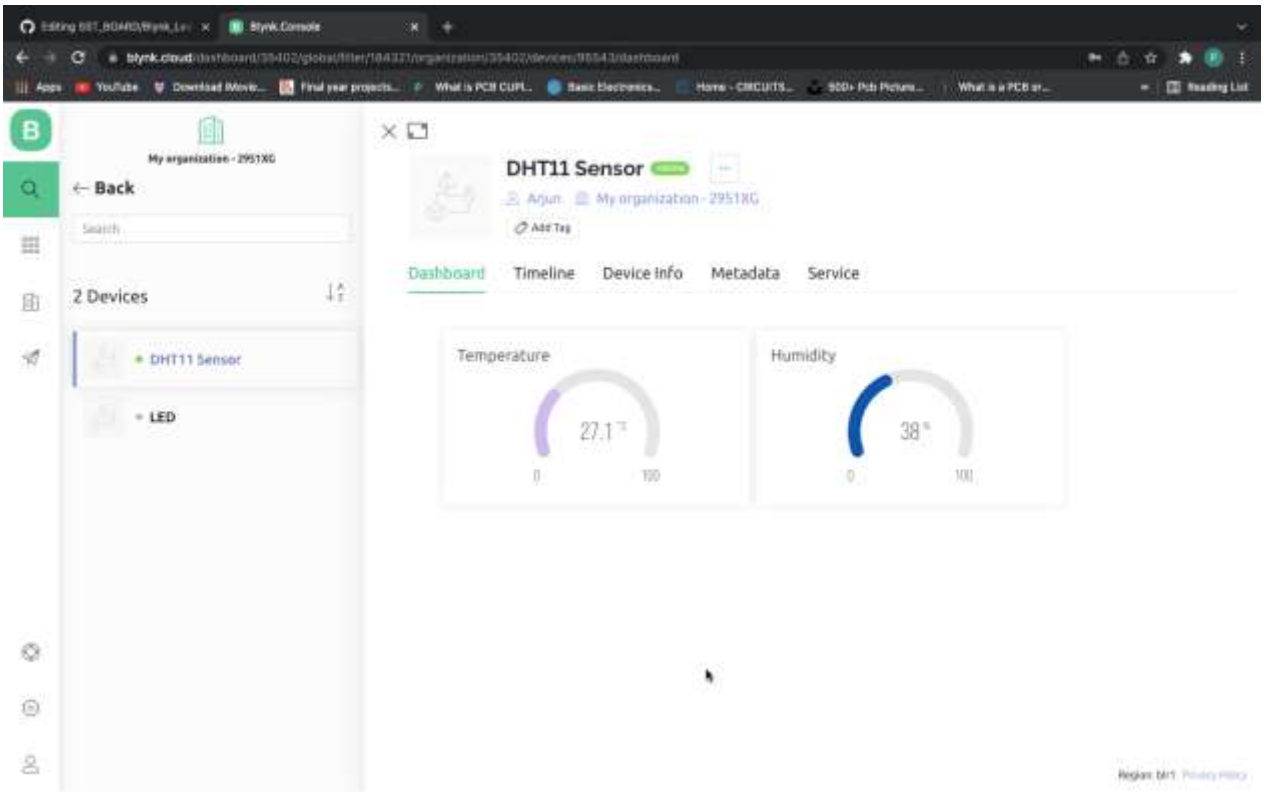
```
#define BLYNK_TEMPLATE_ID "TMPL3Poa118OP"
#define BLYNK_TEMPLATE_NAME "DHT11second"
#define BLYNK_AUTH_TOKEN "ZVr5K4lcXzn8mqchmgNlxoABfjhDWtKt"

#define DHTPIN 15
#define DHTTYPE DHT11

#define BLYNK_PRINT Serial

#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <DHT_U.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

char auth[] = BLYNK_AUTH_TOKEN;

// Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "mahesh";
char pass[] = "1234";

BlynkTimer timer;
DHT_Unified dht(DHTPIN, DHTTYPE);

// This function is called every time the Virtual Pin 0 state changes


// This function is called every time the device is connected to the Blynk.Cloud
BLYNK_CONNECTED()
```

```
{
  // Change Web Link Button message to "Congratulations!"
  Blynk.setProperty(V3, "offImageUrl", "https://static-image.nyc3.cdn.digitaloceanspaces.com/general/fte/congratulations.png");
  Blynk.setProperty(V3, "onImageUrl",  "https://static-image.nyc3.cdn.digitaloceanspaces.com/general/fte/congratulations_pressed.png");
  Blynk.setProperty(V3, "url", "https://docs.blynk.io/en/getting-started/what-do-i-need-to-blynk/how-quickstart-device-was-made");
}

// This function sends Arduino's uptime every second to Virtual Pin 2.
void myTimerEvent()
{

  //Write Code Here:
  sensors_event_t event;
  dht.temperature().getEvent(&event);
  Serial.print("Temperature: ");
  float temp = float(event.temperature);
  Serial.print(event.temperature);
  Blynk.virtualWrite(V1, temp);
  Serial.println("°C");

  dht.humidity().getEvent(&event);
  Serial.print("Relative Humidity: ");
  float hum = float(event.relative_humidity);
  Serial.print(hum);
  Blynk.virtualWrite(V2, hum);
  Serial.println("%");
  Serial.println("\n------------------------------");

}
void setup()
{

  // Debug console
  Serial.begin(115200);
  dht.begin();
  Blynk.begin(auth, ssid, pass);

  // You can also specify server:
  //Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
  //Blynk.begin(auth, ssid, pass, IPAddress(192,168,1,100), 8080);

  // Setup a function to be called every second
  timer.setInterval(1000L, myTimerEvent);
}
```

```
void loop()
{
  Blynk.run();
  timer.run();

  // You can inject your own code or combine it with other sketches.
  // Check other examples on how to communicate with Blynk. Remember
  // to avoid delay() function!
}
```

Sample Output:

Realtime monitoring of DHT11 Temperature and Humidity values on Blynk Dashboard:



Work Sheet

Signature of Staff In-charge with date

SAMPLE VIVA QUESTIONS

1. What is the function of an LED in the experiment?
2. What does GPIO stand for, and what is its purpose?
3. Why is a resistor connected in series with the LED?
4. What is the significance of the pinMode() function in the program?
5. What is the difference between HIGH and LOW in the context of GPIO pins?
6. How does the ESP32 provide power to the LED?
7. What is the purpose of the delay() function in the code?
8. How can you change the blinking interval of the LED?
9. What would happen if you did not connect a resistor to the LED?
10. What are some real-world applications of controlling an LED with a microcontroller like ESP32?
11. What sensor is commonly used to measure temperature and humidity in this experiment?
12. What is the function of the ESP32 in this experiment?
13. What is the role of the heat index in this experiment?
14. Which library is required to interface the DHT sensor with the ESP32?
15. How is the sensor connected to the ESP32?
16. What is the purpose of the Serial.begin() function in the program?
17. Why do we use a pull-up resistor with the DHT sensor?
18. How does the ESP32 calculate the heat index?
19. What unit is used for temperature and humidity in this experiment?
20. What are some real-world applications of this experiment?
21. What is the principle of operation of an ultrasonic sensor?
22. What are the main pins of the ultrasonic sensor used in this experiment?
23. What is the purpose of the TRIG pin in the ultrasonic sensor?
24. How does the ECHO pin work in the ultrasonic sensor?
25. What formula is used to calculate the distance to the object?
26. What is the speed of sound used in this calculation?
27. Why do we divide the calculated time by 2 in the formula?
28. What is the role of the digitalWrite() and digitalRead() functions in the program?
29. What unit is typically used to display the distance measured by the ultrasonic sensor?
30. What are some real-world applications of ultrasonic distance measurement?
31. What is the principle of a soil moisture sensor?
32. What are the main components of a soil moisture sensor module?
33. What pins are used to connect the soil moisture sensor to the ESP32?
34. What type of signal does the soil moisture sensor output?
35. Why do we use the analogRead() function in this experiment?
36. What is the range of analog values that the ESP32 can read?
37. How does the soil moisture sensor differentiate between wet and dry soil?
38. What is the purpose of the Serial.begin() function in the program?
39. What are the practical applications of measuring soil moisture?
40. What are the limitations of soil moisture sensors?
41. What is the working principle of an IR sensor?
42. What are the main components of an IR sensor module?

43. How does the IR sensor detect motion?
44. What pins are used to connect the IR sensor to an ESP32 or microcontroller?
45. What is the role of the buzzer in this experiment?
46. Which function is used to read the sensor output in the program?
47. What does the HIGH or LOW state from the IR sensor indicate?
48. What function is used to turn the buzzer on or off in the program?
49. What are some real-world applications of IR motion detection?
50. What are the limitations of IR sensors for motion detection?
51. What is the working principle of a pulse sensor?
52. What does BPM stand for, and how is it calculated?
53. What are the key pins of the pulse sensor, and how are they connected to the ESP32?
54. What function is used to read the output of the pulse sensor in the ESP32 program?
55. Why is filtering or signal smoothing often applied to the pulse sensor data?
56. How is the heart rate derived from the pulse sensor data?
57. What is the role of the Serial.begin() function in this experiment?
58. What are the normal BPM ranges for adults?
59. What are some common applications of pulse sensors?
60. What are some challenges in accurately measuring BPM using a pulse sensor?
61. What does it mean to configure an ESP32 as an Access Point (AP)?
62. What is the default IP address assigned to the ESP32 in AP mode?
63. Which library is used to enable Wi-Fi functionality in the ESP32?
64. What function is used to set the ESP32 as an access point?
65. What parameters are required to set up the ESP32 as an AP?
66. How can you make the ESP32 access point open (no password)?
67. What is the maximum number of clients that can connect to the ESP32 in AP mode?
68. How do you check if a client is connected to the ESP32 access point?
69. What are the practical applications of using an ESP32 as an access point?
70. What are the advantages of using an ESP32 as an AP over connecting it to an existing Wi-Fi network?
71. What is the purpose of scanning Wi-Fi networks with an ESP32?
72. Which function is used to initiate a Wi-Fi network scan in ESP32?
73. What does RSSI stand for, and what does it indicate?
74. What is the typical range of RSSI values for Wi-Fi signals?
75. How can you display the list of available networks and their strengths on the serial monitor?
76. What does the term SSID stand for, and what is its significance?
77. How can you determine whether a Wi-Fi network is secured or open?
78. What is the purpose of the WiFi.mode() function in this experiment?
79. What are some real-world applications of Wi-Fi network scanning?
80. What factors can affect the RSSI value of a Wi-Fi network?
81. What is the difference between Station mode and Access Point mode?
82. Why is RSSI usually a negative value?
83. What are common causes of Wi-Fi instability?
84. What is the role of URLs in device control?
85. Difference between client and server in IoT?
86. Difference between analogRead() and digitalRead() ?